

PROPOSITION DE PROJET ET GRAND LIVRE TECHNIQUE

OpenDerja_TN

Un corpus écrit ouvert et équilibré par région du derja tunisien — où les Tunisiens écrivent, corrigent et confirment leur propre langue, et où ce qui en sort est citable, téléchargeable, et encore bien vivant dans cinq ans.

Firas Lamouchi · projet solo, open-source et ouvert à la contribution publique une fois le MVP livré

Planification commencée le **14 avril 2026** · développement commencé le **28 mai 2026** · mis à jour le **25 août 2026**

SOMMAIRE

Ce qu'il y a là-dedans

L'argumentaire, puis la machine, puis ce que je n'ai pas encore résolu.

01	La proposition	11	Assistance automatique — reportée
02	Infrastructure et coût	12	Ingestion et scraping
03	Sécurité et abus	13	Standardisation, translittération, appariement
04	Juridique, licences, consentement	14	Le traducteur
05	Ce que je n'ai pas résolu	15	Frontend et système de design
06	Quatre régions, six axes	16	Chaque page, spécifiée
07	Modèle de données	17	Le tableau de bord admin
08	Le moteur de tâches	18	L'API publique
09	Comptes et permissions	19	Export et publication
10	Architecture backend	20	Séquence de construction

Comment lire ce document. Le §1 est l'argumentaire de vente. Les §2–5 sont le test de réalité pratique — coût, sécurité, assise juridique, et les problèmes ouverts — pour que vous sachiez à quoi vous vous engagez vraiment avant que le schéma ne commence. Les §6–19 sont la référence de construction : le modèle de données, les règles, les décisions déjà prises pour ne pas avoir à les reprendre à 2h du matin. Le §20 est l'ordre dans lequel je construis. Le document compagnon Architecture & Engineering Blueprint couvre la façon dont le code a le droit d'être structuré, et VOICE.md couvre la façon dont le site parle aux gens — un travail différent de la façon dont ce document me parle à moi.

Ce document se situe entre deux autres que je n'ai pas encore écrits : une version non technique pour les gens qui veulent juste comprendre ce que c'est et pourquoi, et une version orientée développeurs pour quiconque veut rejoindre le projet et construire. Celui-ci s'adresse aux deux publics à la fois, ce qui explique pourquoi il se lit comme il se lit.

Rien ici n'est définitif. C'est le plan tel que je le comprends après y avoir réfléchi pendant environ deux mois. Je sais déjà qu'une partie se révélera fausse une fois que je serai vraiment en train de construire, et qu'une autre partie changera une fois le dépôt public et que d'autres personnes commenceront à en contester des morceaux. C'est attendu, ce n'est pas un échec du plan — il sera mis à jour au prochain grand jalon, pas figé dans le marbre dès maintenant.

La proposition

Ce que c'est, ce que ce n'est pas, et pourquoi je le fais quand même.

Une plateforme qui continue de tourner, sous licence ouverte, où les Tunisiens écrivent, corrigent et confirment leur propre derja régional. Il en sort un corpus écrit et un lexique — équilibrés par région, organisés en couches de granularité, rattachés à de vraies attestations, et disponibles au téléchargement, à la navigation ou à l'interrogation.

1.1 Ce que c'est

- **Texte d'abord, voix par conception.** La plateforme tourne sur du texte dès le premier jour et porte l'attestation vocale pour une mise à jour future — pas greffée après coup. Le modèle de consentement, le schéma et les tables qui vont avec (§7.11) existent dès la première migration, avant même le premier enregistrement collecté.
- **Équilibré par région, exprès.** La couverture par région est un chiffre que je suis, que je publie et pour lequel je recrute — pas quelque chose que j'espère voir s'arranger tout seul.
- **Vivant, pas archivé.** Les corpus universitaires s'arrêtent le jour où l'article est publié. Celui-ci continue de grandir.
- **Conforme CODA.** Utilise les conventions CODA-TUN et NOTA plutôt que d'inventer encore un autre standard orthographique.
- **Utilisable, pas seulement téléchargeable.** Le corpus est exposé comme un outil que les gens ont une raison d'ouvrir, pas seulement un fichier que les chercheurs ont une raison de citer.

1.2 Ce que ce n'est pas

Le dire à voix haute n'est pas de la modestie. C'est ce qui empêche un chercheur qui connaît le terrain de fermer l'onglet dans les trente premières secondes.

- Pas la première standardisation de l'arabe tunisien. CODA-TUN (LREC 2014) est arrivé le premier ; NOTA l'a remplacé.
- Pas le premier corpus tunisien. Tunisiya, TARIC, TSAC, TEC, TunSwitch, TEDxTN existent déjà.
- Pas la première plateforme de crowdsourcing pour un dialecte arabe. MyVoice/QCRI, DARIJA-C, KALAM'DZ sont arrivés avant.
- Pas en concurrence avec des institutions qui ont financement et personnel.

ALORS QU'EST-CE QUI RESTE

Aucune plateforme existante ne combine tout ça au même endroit : **le texte maintenant et la voix par conception plus tard ; la confirmation communautaire plutôt que la parole d'une seule autorité ; la région et l'époque taguées ensemble, si bien que le vieux derja se retrouve à côté des quatre régions d'aujourd'hui au lieu d'être une archive à part ; un traducteur qui fonctionne vraiment, pas juste un fichier à télécharger ; des données ouvertes que n'importe qui peut récupérer ; et un lieu commun pour publier et discuter de la recherche elle-même.** Chacun de ces éléments existe quelque part, isolément. Nulle part ils n'existent ensemble, entretenus en continu, comme un seul service ouvert. C'est tout l'argumentaire.

1.3 Ce qui compte comme réussite

NIVEAU	À QUOI ÇA RESSEMBLE
Minimum	Un jeu de données derja écrit publié, citable, sous licence ouverte, tagué par région, qui n'existait pas avant, avec des statistiques de couverture honnêtes — et un outil public construit dessus qu'un Tunisien non technique peut utiliser sans rien avoir à lire d'abord.
Bien	Des gens qui contribuent depuis les quatre régions sans que j'aie à pousser. Des gens qui arrivent par le traducteur et qui restent. D'autres chercheurs qui utilisent les données. Une relation de travail avec un groupe de TAL arabe établi.
Solide	Cité dans des travaux de TAL tunisien. Un classificateur régional et un service de mémoire de traduction que les gens utilisent vraiment. Assez de soutien pour que la maintenance ne repose plus seulement sur moi, pour toujours.

1.4 Pourquoi je le construis

C'est un effort à but non lucratif, par conception, depuis la première ligne de code. Pas d'entreprise, pas de parts, aucun projet de faire un jour payer l'accès, rien ici construit pour être vendu. Une ressource linguistique d'intérêt public ne mérite pas de mur payant.

Douze millions de personnes écrivent cette langue tous les jours — dans des textos, dans des commentaires, au milieu de phrases en français — et il n'en existe presque aucune trace correcte. Ma grand-mère a un mot pour désigner quelque chose, dans le nord-ouest, que je n'ai entendu nulle part ailleurs, et quand elle arrêtera de le dire, il aura disparu. Personne ne l'a écrit. Ça arrive tout le temps, et personne ne s'en occupe.

« Quelqu'un devrait écrire ça » est vrai depuis des décennies. Je préfère être celui qui en aura fait une partie. Même la version minimale laisse derrière elle un jeu de données qui n'existait pas et un outil que les gens peuvent utiliser — c'est une vraie chose accomplie, qui tient debout toute seule même si rien après la phase 2 n'arrive jamais.

L'autre raison est tout aussi réelle : c'est un vrai travail d'ingénierie dans un domaine qui me tient à cœur, et j'ai envie de l'avoir construit. Les deux raisons tiennent en même temps.

Les phases sont dimensionnées pour que la valeur survive même si les suivantes n'arrivent jamais. Si je m'essouffle, ça laisse quelque chose de fini, pas quelque chose d'abandonné à moitié.

Infrastructure et coût

Pas encore décidé, exprès — et une décision bien plus facile sans modèle sur la machine.

OÙ EN EST L'HÉBERGEMENT

L'hébergement est encore indécis. Cette section sera mise à jour une fois la décision prise. Ce que ce projet devient détermine ce dont il a besoin pour tourner, donc les specs restent ouvertes jusqu'à ce que la construction soit plus avancée. Le déploiement est un problème de phase 2 (§20). Rien dans les phases 1 à 3 n'en dépend.

2.1 Hébergement

Fournisseur indécis — sera mis à jour ici une fois choisi. Pas de local physique permanent, donc l'auto-hébergement matériel n'est une option dans aucune version de ce projet ; la décision se joue entre une poignée de petits fournisseurs VPS européens ou compatibles ARM, dimensionnée contre l'usage mesuré une fois la construction plus avancée plutôt que devinée maintenant.

2.2 Forme du déploiement

```
VPS (fournisseur à définir — même topologie quel qu'il soit)
├─ cloudflared    tunnel — aucun port entrant ouvert sur la machine (§3.1)
├─ caddy          TLS, reverse proxy, limitation de débit, routage par nom d'hôte
├─ frontend       Next.js (site public + tableaux de bord)
├─ backend        NestJS — frontière de l'API, chaîne de guards (§10.1)
├─ worker         tâches de fond, concurrence plafonnée (§10.3)
├─ postgres       instance unique — tout l'actif du projet
├─ pgbouncer      pooling de connexions devant postgres (§10.2)
├─ redis          cache + sessions seulement, pas la file de tâches (§10.3)
├─ nlp            Python, health-check seulement, phase 1 (§11)
├─ processing     Rust, health-check seulement, phase 1 (§7.11)
└─ backups        pg_dump nocturne, chiffré, -> stockage objet hors machine
```

Dix conteneurs au lieu de quatre — le backend séparé dans son propre processus, pgBouncer et Redis ajoutés, et les deux services des phases futures qui apparaissent tôt comme des coquilles vides. Tout reste multi-architecture, donc ARM contre x86 n'est pas une décision à prendre à l'avance. Des limites mémoire explicites restent sur chacun d'eux, et ça compte encore plus maintenant qu'à quatre services, pas moins.

Les deux services health-check-seulement et Redis sont vraiment légers — aucun d'eux ne détient de vraies données ni ne fait de vrai travail pour l'instant, donc le bond en mémoire dû à leur ajout est petit comparé à ce que coûterait un vrai modèle de ML en production (§11). pgBouncer lui-même est bon marché ; ce contre quoi il protège, c'est Postgres qui manque de connexions, pas de mémoire. Le chiffre qui compte vraiment reste ce que sera le VPS (§2.1) — cette forme tient confortablement dans le petit palier, ce n'est juste plus le diagramme à quatre boîtes d'avant.

2.3 Sauvegardes

Le nom de domaine n'est pas encore choisi — ça se réglera une fois la construction terminée et une fois que je saurai ce qui tourne vraiment.

LES SAUVEGARDES, QUI NE SONT PAS NÉGOCIABLES

Le corpus est la seule chose irremplaçable ici. Je peux réécrire le code en un week-end. Je ne peux pas réécrire des mois d'annotation communautaire.

- `pg_dump` nocturne, compressé et **chiffré avant de quitter la machine** — le dump contient des emails, des valeurs `ip_hash` et des identifiants de session, exactement ce que le §4.3 s'efforce de protéger, donc ça ne peut pas voyager en clair.
- Export hebdomadaire d'un instantané vers Hugging Face — une publication publique et une sauvegarde hors site en une seule opération
- Instantanés côté fournisseur activés (quelques euros par mois, clairement rentable)
- Restauration testée trimestriellement — un workflow `verify-backup-restore.yml` planifié (§10.0) le fait automatiquement, pas un rappel dans mon agenda que je peux repousser. Une sauvegarde jamais testée n'est pas une sauvegarde.
- Toute la pile dans `docker-compose.yml`, pour que changer de fournisseur prenne moins d'une heure

2.4 Coût

Pas encore chiffré. Ça ne sert à rien de chiffrer une machine avant de savoir ce qui va vraiment tourner dessus — le coût se réglera au lancement, une fois l'hébergement et le reste de cette forme vraiment décidés, pas devinés maintenant.

2.5 Stockage objet — encore indécis

Les sauvegardes et les exports d'instantanés ont besoin d'un endroit qui ne soit pas le serveur d'application, et ça reste indécis en même temps que l'hébergement. Le besoin d'aujourd'hui est petit — dumps nocturnes, exports hebdomadaires, confortablement dans n'importe quel palier gratuit. Le besoin si la voix arrive (§7.11) est une question complètement différente, et c'est elle qui devrait trancher plutôt que le besoin actuel.

L'axe qui compte pour l'audio n'est pas le prix du stockage — bon marché partout. C'est l'**egress**, parce que servir les extraits audio en retour coûte de l'argent réel et les fournisseurs varient d'un ordre de grandeur là-dessus. Trancher ça sur les besoins de sauvegarde d'aujourd'hui reviendrait à trancher sur le mauvais critère.

CANDIDAT	POSITION
Cloudflare R2	Zéro frais d'egress. Le candidat par défaut dès que la voix est sur la table.
Backblaze B2	Stockage bon marché, l'egress coûte quelque chose. Très bien pour les sauvegardes.
Stockage natif du fournisseur	Le moins cher s'il vient du même fournisseur que le VPS de toute façon, pas toujours compatible S3 de la même façon.
S3 classique	Fonctionne, le plus cher en egress, aucune raison de le choisir en premier.

Décidé en phase 2 en même temps que l'hébergement, dimensionné pour les sauvegardes, revisité avant tout travail sur l'audio.

2.6 Email

Les invitations aux relecteurs et la vérification d'email ont besoin d'une livraison fiable, et un email auto-hébergé depuis une IP de VPS toute neuve finit systématiquement dans les spams — ça mord fort le jour du déploiement si ce n'est pas anticipé. Resend est le principal, avec SMTP en repli plutôt qu'un second fournisseur à configurer (§10.0, §9.11) : environ 0–10 €/mois à ce volume.

2.7 Ce que je ne fais pas

AWS / Azure

Kubernetes

Postgres géré

CDN

multi-région

instances GPU

inférence permanente

Si je veux de l'expérience avec un fournisseur cloud pour une certification, c'est un bac à sable séparé sur palier gratuit, avec des ressources jetables. Ça ne passe pas sur la facture ni la disponibilité de ce projet.

03

Sécurité et abus

Un modèle de menace dimensionné à ce qui va vraiment arriver, pas à ce qui sonne impressionnant.

3.1 Ce qui m'inquiète vraiment

MENACE	PROBABILITÉ	CE QUI L'ARRÊTE
Soumissions spam	Élevée	Limites de débit par session et ip_hash ; le classement ne compte que les contributions approuvées donc le spam ne rapporte rien ; annulation en masse par session
Empoisonnement délibéré des données	Moyenne	Rien ne devient canonique sur la parole d'une seule personne ; le consensus a besoin de plusieurs annotateurs ; tout est réversible
Contenu offensant ou personnel publié publiquement	Moyenne	Bouton de signalement sur chaque permalien (§16.4), qui alimente la file de relecture des corrections
Scraper une source qui n'est en fait pas autorisée	Moyenne	Lecture des CGU obligatoire avant qu'une source ne soit activée, pas seulement le robots.txt (§12.5)
La file d'attente qui affame le frontend	Moyenne	Intervalle de sondage et concurrence pg-boss plafonnés ; travail en masse en heures creuses (§10.3)
Les commentaires de publications qui deviennent une charge de modération sans fin	Moyenne	Retenus pour approbation en dessous de la confiance 1, et la fonctionnalité est désactivée si elle coûte plus qu'elle ne rapporte (§7.10)
Compromission d'un compte relecteur	Faible	Argon2id, sessions httpOnly, 2FA sur l'admin, journal d'audit complet (§9.7)
Accès entrant non autorisé	Faible	Éliminé structurellement, pas atténué — aucun port ouvert sur le VPS, point. Tout arrive via Cloudflare Tunnel (§10.0) ; il n'y a même pas de port à scanner ou à attaquer.
Perte de données	Faible, fatale	§2.3

MENACE	PROBABILITÉ	CE QUI L'ARRÊTE
Contestation juridique sur du contenu scrapé ou stocké	Faible à moyenne, revue à la hausse	Provenance et licence par enregistrement ; une liste de sources resserrée après la correction du §12 ; mise en quarantaine et purge des sources
Plainte pour rétention de données au titre du RGPD	Faible	Routine d'anonymisation testée qui nettoie les identifiants et rend le texte orphelin (§4.3)

3.2 Base de référence

Limitation de débit au niveau de Caddy, par IP pour l'anonyme et par clé pour l'API. Toutes les entrées validées par des schémas zod partagés à la frontière. Requêtes paramétrées uniquement — Prisma s'en charge, aucune interpolation de chaîne brute nulle part. En-têtes CSP, pas de scripts en ligne. Secrets dans l'environnement, jamais commités, tournés à la moindre suspicion. Sous-domaine admin restreint par IP là où je peux le gérer. Dépendances figées, scan de vulnérabilités actif.

Juridique, licences, consentement

Deux licences, choisies séparément, et une case à cocher qu'on ne peut pas ajouter plus tard.

4.1 Deux licences

ACTIF	LICENCE	RAISONNEMENT
Code	MIT ou Apache-2.0	Permissive. L'adoption compte plus que le contrôle.
Jeu de données	CC-BY-SA ou CC-BY-NC — pas encore tranché	Ce n'est pas une question de protéger des revenus — il n'y en a pas. C'est d'empêcher quelqu'un d'autre de prendre ce que les contributeurs ont construit gratuitement, de l'enfermer, et de le leur revendre. Le partage à l'identique ou le non-commercial ferment tous les deux cette porte ; le permissif pur ne la ferme pas. Ça demande une vraie décision, pas un choix par défaut.

Le crédit du code vit séparément du crédit du corpus. `docs/CONTRIBUTORS.md` nomme les gens qui travaillent sur le code ; le classement public `/contributors` (§16.1) crédite les gens qui travaillent sur le corpus. Contribution différente, public différent — fusionner les deux enterrait le groupe bien plus large qui fait le vrai travail de données sous une poignée d'auteurs de commits.

4.2 Consentement

Une case à cocher explicite à la soumission : ce contenu peut entrer dans un jeu de données sous licence ouverte, usage commercial inclus. Deux lignes d'interface. Sans ça, aucun contenu contribué n'est jamais utilisable commercialement en toute sécurité, et le consentement ne peut pas être recueilli plus tard une fois que les contributeurs sont passés à autre chose.

Ce consentement couvre l'enregistrement dans son ensemble, pas seulement le texte brut — y compris quand un relecteur attribue plus tard une origine au mot d'un contributeur, ou le tague vulgaire, archaïque, ou régional. Ce sont des jugements posés par quelqu'un d'autre, publiés à côté de la contribution, et « ce contenu peut entrer dans un jeu de données sous licence ouverte » est écrit pour désigner l'enregistrement annoté, pas seulement la chaîne brute que quelqu'un a tapée. Ça vaut le coup de le rendre explicite plutôt que de le laisser en hypothèse.

LA CLAUSE VOIX EMBARQUE DÈS LA PREMIÈRE CONTRIBUTION, DES ANNÉES À L'AVANCE

La case à cocher porte une clause voix séparée dès le premier jour, même si l'enregistrement n'existe pas et n'existera pas avant longtemps (§7.11). Le consentement texte ne couvre pas la voix, et il ne peut pas être recueilli rétroactivement — le même argument, deux fois plus dur, parce qu'un enregistrement identifie une personne d'une façon qu'une phrase ne fait pas. Le défaut est texte seulement. La voix est un second opt-in, explicite.

4.3 Révocation — la seule chose qui supprime pour de vrai

DEUX RÈGLES DE CE DOCUMENT ENTRENT EN COLLISION ICI

Le §17.2 dit que rien n'est supprimé pour de vrai — suppression douce plus une entrée d'audit, parce que le jeu de données est tout l'actif du projet. Mais héberger dans l'UE place ça sous le RGPD. Si quelqu'un révoque son consentement, garder son `ip_hash` et son email est un problème qu'un simple drapeau de suppression douce ne règle pas. Une ligne marquée supprimée reste une ligne conservée.

La sortie consiste à séparer deux choses qu'on traitait comme une seule : **la personne** et **la langue**. Les données personnelles peuvent vraiment être effacées. La contribution linguistique n'est plus une donnée personnelle une fois qu'elle n'est plus rattachée à personne.

```

lors d'une révocation de consentement (tâche horaire : anonymise_revoked)
Nettoyage DUR, irréversible, aucune copie d'audit conservée :
  submission_meta.contributor_email, contributor_name, ip_hash, session_id -> null
  users.email, users.display_name -> null (s'ils avaient un compte)
  audit_log.ip_hash -> null pour cet acteur ; audit_log.diff nettoyé
CONSERVÉ, définitivement orphelin :
  le texte de l'élément du corpus, ses tags/tokens/liens, ses traductions, sa région/époque/registre,
  son dataset_split
ENREGISTRÉ :
  documents.rights_status -> 'revoked' ; une entrée d'audit SANS identifiant — action, horodatage, dé

```

- Le texte reste, la personne s'en va. Les instantanés déjà publiés ne sont pas affectés — ils n'ont jamais porté d'identifiants (§19.2).
- Irréversible, et c'est le but : réservé aux admins, confirmation tapée, limité en fréquence.
- La suppression totale est honorée sur demande ; la révocation par défaut se limite à l'anonymisation.

La voix est l'exception de l'exception. Quand des enregistrements existeront (§7.11), révoquer le consentement voix supprime carrément l'objet et la ligne media — impossible de se rabattre sur l'anonymisation comme pour le texte, parce qu'un enregistrement est quasi-biométrique. C'est le deuxième et dernier endroit où quelque chose est vraiment détruit, conçu maintenant, avant même qu'un seul enregistrement n'existe.

4.4 Contenu scrapé

Stocké avec sa source et son drapeau de licence, exclu par défaut du palier commercialement propre. Public n'est pas une licence — le texte scrapé est réservé à un usage recherche sauf preuve du contraire (§12.2). Les demandes de retrait sont honorées via le mécanisme de signalement (§16.4) et la mise en quarantaine des sources. Aucune donnée personnelle issue de sources sociales — les identifiants d'auteur sont retirés à l'ingestion.

4.5 Créditer les travaux antérieurs

CODA-TUN, NOTA et tout corpus que je réutilise sont cités sur la page méthodologie et dans la fiche du jeu de données.

05

Ce que je n'ai pas résolu

Écrit noir sur blanc, parce qu'un risque qu'on n'a pas nommé n'est pas un risque qu'on gère.

#	PROBLÈME	OÙ ÇA EN EST
1	Hébergement indécis. Sera mis à jour ici une fois la décision prise.	Laissé ouvert jusqu'à ce que la construction soit finie et que je puisse mesurer. Bien moins d'enjeu sans modèle sur la machine.
2	Licence du jeu de données. Permissive, partage à l'identique, ou non-commerciale.	Doit être tranchée avant la première publication publique.
3	Les frontières régionales sont une hypothèse de travail, pas de la dialectologie établie.	Le tagging multi-région (§6.4) laisse les données me montrer où les frontières tombent vraiment.
4	Recrutement de relecteurs. Quatre régions ont besoin de relecteurs venant de quatre régions et je n'ai aucun réseau pour ça.	Non résolu, le risque non technique le plus sérieux. Le routage (§8.5) limite les dégâts, ne règle pas le problème.
5	Vérité terrain régionale. La région auto-déclarée est le seul signal, et il n'est pas fiable — les gens déménagent.	Atténué en demandant l'origine séparément de la résidence. Pas résolu.
6	Les livres et l'imprimé ont besoin d'OCR arabe avec gestion de mise en page RTL.	Reporté. La voie principale vers era: historical.
7	Rester en vie. Dépend de la continuité de l'effort non rémunéré d'une seule personne.	Ne peut pas être pleinement atténué. La discipline de périmètre est la seule vraie défense.
8	Le sahel comme une seule région — Sfax pourrait être assez distinct de Sousse pour être séparé.	Surveiller le désaccord entre annotateurs, séparer seulement si les données l'exigent.
9	Fatigue de la file de relecture sans filtre de dialecte — la fuite de bruit passe d'environ 20–30 % à 40–50 % de ce qui atteint la relecture (§12.3).	Compromis accepté pour l'instant. Petits lots, désactiver les sources à faible rendement. Faire venir un filtre plus tôt que la phase 4 reste une question ouverte — à revoir si la file dépasse les relecteurs.
10	Durabilité de l'IP du scraper — un seul serveur, une seule adresse, les sources HTML finiront par la bannir.	Réduit par un budget de proxys et du backoff (§12.5). Pas éliminé.
11	Démarrage à froid du traducteur — tant que le lexique n'a pas de couverture, la plupart des recherches renvoient « non attesté » et ça a l'air cassé.	Amorcer avec des entrées écrites à la main dans les quatre régions avant le lancement ; présenter chaque échec comme une invitation, pas comme un raté (§14.4).
12	Stockage objet indécis , et le critère décisif — l'egress pour la voix — est une préoccupation de phase 5.	Choisir quelque chose pour les sauvegardes maintenant, revoir avant l'audio (§2.5).
13	Le consentement voix ne peut pas être ajouté après coup , et la révocation voix est la deuxième exception au principe de non-suppression.	La clause de consentement embarque dès la première contribution (§4.2). Le chemin de suppression est conçu avant même qu'un enregistrement n'existe (§4.3).
14	Les commentaires de publications sont un travail sans limite — la seule surface qui peut générer de l'effort qui n'est pas de l'effort de corpus.	Retenus pour approbation en dessous de la confiance 1, désactivés entièrement si ça coûte plus que ça ne rapporte (§7.10).

Côté commercial. Je ne compte pas dessus — voir §1, je n'essaie même pas. Tout ce qui est après la phase 2 est dimensionné pour que la valeur survive même si le reste de cette liste reste exactement aussi longue pour toujours.

Quatre régions, six axes

Se tromper là-dessus fait que tout ce qui suit est faux avec.

6.1 Régions — plates, sans sous-régions

VALEUR	COUVRE
northwest	Le Kef, Jendouba, Siliana, Kasserine, les hauts plateaux de l'ouest
north	Tunis, Bizerte, Nabeul, Ariana, Manouba, Ben Arous
sahel	Sousse, Monastir, Mahdia, Sfax, le centre côtier
south	Gabès, Médenine, Tataouine, Tozeur, Kébili, Douz, Gafsa

Quatre valeurs, pas de sous-régions tant qu'il n'y a pas de données pour les justifier. La structure `subregion` existe dès le premier jour et reste vide — l'ajouter plus tard est une migration sur une seule colonne, pas un changement de schéma.

6.2 Six axes indépendants

C'est la décision dont tout le reste dépend. Région, époque et cadre ne sont pas la même sorte de chose et ne doivent jamais être écrasés dans une seule liste.

AXE	VALEURS	REMARQUE
scope	pan_tunisian · regional	Si c'est marqué régionalement ou non, pas à quel point c'est répandu
region	un ensemble parmi : northwest · north · sahel · south	Un ou plusieurs. Vide quand c'est pan-tunisien. §6.4.
subregion	vide (réservé)	Reporté
era	contemporary · historical	Le « vieux derja » vit ici, pas dans la région
setting	urban · rural · unknown	Traverse les régions, ne les suit pas
register	neutral · formal · vulgar · archaic	Sans ça, le résultat n'est pas exploitable pour grand-chose. Politique d'affichage au §16.4.

« Nord-ouest + rural + historique » est une vraie chose qu'on peut demander. Quatre cases plates ne pourraient pas l'exprimer — et cette combinaison est exactement le genre de matériau que personne n'a collecté, ce qui explique en grande partie pourquoi ce projet existe.

6.3 Granularité — encore un axe séparé

	PAN-TUNISIEN	RÉGIONAL
mot	vocabulaire partagé	un mot du nord-ouest pour un objet du quotidien
phrase	tournure utilisée partout	une construction qui trahit d'où on vient
paragraphe	texte suivi neutre	discours suivi avec marquage régional

6.4 La région est un ensemble, parce que le dialecte est un continuum

Les isoglosses ne respectent pas les cases. Un mot rural peut être partagé entre le nord-ouest et le sud tout en étant inconnu à Tunis. Un vocabulaire côtier peut couvrir le nord et le sahel et s'arrêter là. Ni l'un ni l'autre n'est pan-tunisien, et ni l'un ni

l'autre n'appartient à une seule région.

Avec une colonne à valeur unique, chaque cas comme ça n'a que deux mauvaises options : choisir une région et jeter une vraie attestation, ou dire pan-tunisien et jeter le fait même que c'est marqué. Les deux abîment les chiffres de couverture, qui sont précisément ce que ce projet prétend bien faire. Donc :

- **scope, c'est le marquage.** Est-ce que ça porte un signal régional, oui ou non ? C'est vraiment binaire.
- **region, c'est l'étendue.** Quelles régions l'attestent, une à quatre. Les quatre à la fois n'est pas la même affirmation que pan-tunisien — « utilisé partout et marqué dans chacune » diffère de « non marqué ».

Il n'y a donc de colonne région à valeur unique nulle part. Les éléments du corpus portent plusieurs lignes `tag` sur le même intervalle (§7.3) ; les variantes du lexique utilisent une table `lexicon_variant_regions` (§7.5), qui donne aussi à chaque région son propre compte d'attestations.

Ça permet aussi à la question des frontières régionales (§5, problème 3) de répondre d'elle-même. Si nord-ouest et sud continuent d'être tagués ensemble, c'est une preuve de l'endroit où la frontière tombe vraiment. Une colonne à valeur unique aurait détruit cette preuve dès l'insertion, à chaque fois.

Modèle de données

Une seule structure avec annotation au niveau des intervalles, parce que les niveaux se contaminent entre eux.

7.1 Le problème de jointure

Une phrase est souvent marquée comme régionale par un seul mot en elle. Un paragraphe peut citer une variété dans laquelle il n'est pas écrit. Trois tables indépendantes ne peuvent pas contenir ça, donc c'est une seule structure avec des annotations rattachées à des intervalles de caractères.

7.2 Documents et éléments du corpus

```
documents
  id, source_id fk -> sources
  external_ref      text          -- id vidéo, url, isbn ; null pour les contributions
  title, raw_body   text          -- raw_body est l'original, jamais retouché ensuite
  region_hint       enum          -- signal faible venant de la source, pas un fait ; à valeur unique e
  era_hint, license enum
  rights_status     enum          -- granted | revoked | not_applicable (§4.3)
  ingested_at       timestamptz

corpus_items
  id, document_id fk, parent_id fk -- parent null au niveau racine
  unit              enum          -- paragraph | sentence | phrase          (pas de token)
  position          int           -- ordre au sein du parent
  text              text          -- immuable une fois inséré
  script            enum          -- arabic | arabizi | mixed | latin
  canonical_form     text          -- forme CODA, null tant que non standardisé
  canonical_map      jsonb        -- alignement d'offsets : text -> canonical_form
  match_key          text          -- normalisé pour la recherche, jamais affiché (§13.5)
  rule_version       int
  char_offset        int          -- offset dans le parent, pour reconstruction
  dataset_split      enum          -- train | dev | test ; fixé à l'insertion, jamais modifié
  version            int          -- incrémenté à chaque changement accepté (§8.1)

index (document_id, unit, position)
index (parent_id)
index gin (match_key gin_trgm_ops)
```

Les contributions sont enveloppées dans un document synthétique à un seul élément de corpus, pointant vers la source contribution. C'est délibéré, pas juste soigné : `license`, `rights_status` et la provenance vivent sur `documents` et nulle part ailleurs, donc « est-ce exportable commercialement ? » est une seule requête contre une seule table, quelle que soit l'origine. Un `document_id` nullable aurait signifié copier ces colonnes sur les éléments du corpus et dédoubler cette vérification pour toujours.

`region_hint` reste un `enum` à valeur unique alors même que le §6.4 a fait un vrai effort pour faire de la région un ensemble partout ailleurs. C'est délibéré, pas un oubli : un indice est une supposition sur d'où vient probablement toute une source — une chaîne, la base d'utilisateurs générale d'un forum — avant que quiconque n'ait lu un seul mot. Il a le droit d'être faux et singulier parce que rien en aval ne lui fait confiance ; l'affirmation de région réelle, qui doit bien être un ensemble, vit sur les tags rattachés aux intervalles (§7.3), pas ici.

POURQUOI LES TOKENS NE SONT PAS DES LIGNES ICI

Le mouvement évident serait un troisième niveau sous les phrases. Ça casse dès qu'on standardise quoi que ce soit. Convertir en CODA change le nombre de mots et tous les offsets après la première modification — l'arabizi « *bch nemchi* » devient *باش نمشي*, les clitiques s'étendent, deux tokens en deviennent trois. Chaque ligne de token et chaque annotation qui pointe dessus arrête tranquillement de correspondre. Rien ne plante. Les données se mettent juste à mentir.

Les tokens ont donc leur propre table (tokens), ancrée sur des offsets dans text. text ne change jamais, donc ces offsets restent justes pour toujours. Une table links séparée est l'unique pont entre un token et le lexique — token_id y est unique, donc rien n'empêche qu'un token pointe vers deux entrées de lexique différentes, une contrainte qu'une table partagée unique n'aurait jamais pu exprimer.

La standardisation ne modifie jamais text. Elle écrit une seconde version à côté, plus canonical_map — l'alignement d'offsets que le moteur de règles produit de toute façon (§13.4). N'importe quel intervalle sur l'original se projette sur la forme canonique via cette carte, et comme la carte porte son rule_version, changer les règles régénère les deux ensemble.

7.3 Tokens, liens et tags — sur des intervalles, pas des niveaux

C'est ça qui règle la contamination. Un paragraphe tagué south peut contenir un intervalle de trois mots tagué comme emprunt du nord. Les deux sont vrais, les deux sont stockés, pas de dispute.

Trois tables, pas une seule table polymorphe annotations — la version à table unique est l'endroit où le flou des noms s'était glissé, parce qu'une ligne type: region et une ligne type: token n'ont rien en commun sauf de partager une table. Les séparer s'est révélé être une vraie correction de bug, pas juste cosmétique : c'est ce qui permet de déclarer links.token_id unique, ci-dessous.

```
tokens                                -- un intervalle dans le texte d'un élément du corpus
  id, corpus_item_id fk
  char_start, char_end  int
  surface_text  text    -- le texte littéral du token
  annotator_id fk      -- l'utilisateur 'system' pour la tokenisation automatique
  is_machine    bool
  confidence    numeric(3,2)
  index (corpus_item_id)

links                                -- l'UNIQUE pont entre un token et le lexique
  id, token_id fk UNIQUE      -- un lien par token, jamais deux
  lexicon_entry_id fk, lexicon_variant_id fk null
  annotator_id fk, is_machine bool, confidence numeric(3,2)
  index (lexicon_entry_id)
  index (lexicon_variant_id)

tags                                -- tout le reste : scope, region, era, setting,
                                -- register, code_switch, sense, quality
  id, corpus_item_id fk
  char_start, char_end  int
  kind    enum    -- scope | region | era | setting | register
                -- | code_switch | sense | quality
  value    text
  annotator_id fk      -- l'utilisateur 'system' pour les suggestions automatiques
  is_machine    bool
  confidence    numeric(3,2)

  index (corpus_item_id, kind, char_start, char_end)
  index (annotator_id)
```

Tokeniser est une passe à part entière, qui insère une ligne tokens par token — retokeniser veut dire insérer des lignes, pas reconstruire un arbre. Un tag de région est une ligne *par région* — taguer un intervalle nord-ouest et sud, c'est deux lignes, et

c'est comme ça que la sémantique d'ensemble du §6.4 fonctionne sans colonne tableau.

TROIS CHOSES QUE ÇA DONNE GRATUITEMENT

L'alternance codique — un emprunt français est un intervalle avec `kind: code_switch`, `value: fr`. **Corpus et lexique se rejoignent** — une ligne `links` qui pointe un token vers une entrée de lexique transforme « cette phrase atteste la forme du nord-ouest » en une seule requête. **Le désaccord survit** — deux annotateurs écrivent deux lignes `tags` ; le consensus se calcule à partir d'elles, sans jamais les écraser.

7.4 Consensus — dérivé, jamais destructeur

Indexé par intervalle, pas par élément du corpus, et ça compte plus qu'il n'y paraît. Une phrase porte couramment plusieurs tags du même genre — trois intervalles `code_switch`, deux intervalles avec des identités régionales différentes. (Les liens n'ont pas besoin de ça — un token a au plus un lien, donc il n'y a rien sur quoi trouver un consensus.) Indexer par élément du corpus et genre seulement, et calculer le consensus pour le second intervalle écrase le premier. Plus quelqu'un tague une phrase avec soin, plus la table jette de son travail.

```
tag_consensus          -- matérialisé, rafraîchi à chaque écriture de tag
  corpus_item_id fk, kind enum, char_start int, char_end int
  agreed_values         text[]          -- un ensemble : la région peut faire consensus sur plus d'une val
  agreement            numeric(3,2)    -- 1.0 = unanimité
  annotator_n          int
  needs_adjudication    bool
  computed_at          timestamptz

PRIMARY KEY (corpus_item_id, kind, char_start, char_end)
index (needs_adjudication) where needs_adjudication
```

- **Les tags portant sur tout l'élément ont quand même un intervalle.** `register`, `era`, `scope` sont écrits avec `char_start = 0`, `char_end = length(text)`. Jamais null — Postgres ne traite pas les null comme égaux dans une clé unique, donc un intervalle nullable laisserait passer des doublons exactement là où l'unicité est le but.
- **L'accord se calcule sur un ensemble.** Deux personnes qui disent toutes les deux « nord-ouest et sud » sont d'accord. Une qui dit « nord-ouest » et une qui dit « nord-ouest et sud » sont partiellement d'accord — la lecture honnête, et un signal utile pour la question des frontières.

7.5 Lexique — trois couches

```
lexicon_entries          -- le sens, agnostique à la région
  id, gloss_en, gloss_fr, gloss_msa
  domain enum -- everyday | food | admin | agriculture | kinship | ...
  granularity enum, pos enum, notes

lexicon_variants         -- une réalisation d'une entrée du lexique
  id, lexicon_entry_id fk
  scope, era, setting, register -- axes à valeur unique seulement
  canonical_form text
  match_key text -- §13.5
  confidence numeric(3,2)

lexicon_variant_regions  -- étendue : une ligne par région attestante (§6.4)
  lexicon_variant_id fk ON DELETE CASCADE, region enum
  attestation_count int, confidence numeric(3,2)
  PRIMARY KEY (lexicon_variant_id, region)
  index (region, attestation_count desc)

lexicon_forms            -- une graphie écrite d'une variante
  id, lexicon_variant_id fk, text, script, match_key
  is_canonical bool, frequency int
  unique (lexicon_variant_id, script) where is_canonical
```

Pourquoi trois couches. La question à laquelle le produit existe pour répondre — « voici quatre façons de le dire, laquelle est celle-ci ? » — devient une seule jointure. À plat, on ne peut pas la poser. Trois couches permettent aussi d'exprimer les faux amis : une même graphie qui rejoint deux entrées de lexique via deux variantes différentes. Ça arrive tout le temps, et un modèle plat le corrompt silencieusement.

7.6 Origine des mots

Les strates du derja, c'est son histoire — substrat amazigh, superstrat arabe, puis le français, l'italien, le turc et l'espagnol arrivés par l'occupation, le commerce et les migrations. Enregistrer de quelle strate vient un mot transforme une liste de vocabulaire en document de cette histoire, et c'est la chose la plus intéressante du site pour quiconque n'est pas linguiste. Aucun corpus tunisien existant ne porte de tagging étymologique systématique.

Ça se rattache à `lexicon_entries`, pas à `lexicon_variants` — l'origine est une propriété de l'histoire du mot, pas de la façon dont Le Kef le dit par rapport à Douz. L'exception est réelle mais rare : parfois des régions ont emprunté des mots *différents* pour le même sens à des sources différentes. C'est sans doute deux entrées de lexique, et je préfère laisser le cas rare porter une note plutôt que de dupliquer la colonne sur les variantes pour une minorité d'entrées.

```
lexicon_origins          -- la plupart des entrées du lexique n'en ont pas, et c'est très bien ai
  id, lexicon_entry_id fk ON DELETE CASCADE
  origin enum
  source_form text null -- 'écurie', 'tanwart' — le mot dont ça vient
  source_lang text null
  note text null -- la dérivation, en prose, avec une citation s'il y en a une
  proposed_by fk, status enum -- proposed | confirmed | disputed
  confidence numeric(3,2)
  index (lexicon_entry_id); index (origin) where status = 'confirmed'

origin_enum:
  arabic -- direct, reconnaissable comme arabe standard
  arabic_derived -- vient de l'arabe mais a dérivé en forme ou en sens
  french amazigh italian
  turkish spanish other
  unknown -- explicitement enregistré comme inconnu
```

L'enum vit dans `shared` avec les autres (§10.1). Deux choses à ce sujet :

- **L'italien et le turc sont là dès le début parce qu'on les rencontre vite.** Le tunisien a une vraie strate italienne via la Sicile et les ports, et une strate administrative et culinaire ottomane. Les laisser de côté et les contributeurs les rangeraient sous `other`, et on ne les récupère jamais proprement après.
- **unknown n'est pas la même chose qu'aucune ligne.** Aucune ligne veut dire que personne n'a regardé. Une ligne disant `unknown` veut dire que quelqu'un a regardé et n'a pas pu trancher. Le second cas est une information.

Exemples

STRATE	DERJA	VIENT DE	SENS
amazigh	فكرون fakrun	ifker	tortue
amazigh	نواره nawwara	tanwart	fleur
french	فرشيطة farshiṭa	fourchette	fourchette
french	بيرو birou	bureau	bureau
french	تريسيتي trisiti	électricité	électricité
french	كوري kouri	écurie	écurie
italian	كروسة karrusa	carrozza	charrette
italian	بانكة banka	banca	banque
italian	سيمانة simana	settimana	semaine
turkish	طبسي ṭabsi	tepsi	plateau, plat
turkish	بقلاوة baklawa	baklava	—
arabic_derived	باهي bahi	بلا classique : splendide	a dérivé vers l'ordinaire « bon, bien, OK »

LA MEILLEURE ENTRÉE DE LA PAGE N'EST PAS UN MOT

Le suffixe `-ji` (جي) vient de l'agentif turc `-ci`, et le tunisien l'applique à des racines arabes — قهواجي *gahwaji*, le vendeur de café. Le derja n'a pas seulement emprunté des mots turcs, il a emprunté un bout de grammaire turque et l'utilise encore de façon productive. Ça mérite sa propre note sur `/origins`.

Contestés

DERJA	L'ARGUMENT
سبيطار sbiṭar — hôpital	Français <i>hôpital</i> ou italien <i>ospedale</i> ? Les deux plausibles, les deux défendus.
فيشته fishta — fête	Français <i>fête</i> ou italien <i>festa</i> ?
برشا barsha — beaucoup	Le mot le plus disputé du tunisien. Turc, arabe et d'autres ont tous leurs défenseurs.

Trois entrées contestées à côté des confirmées valent plus que cinquante entrées bien rangées. Elles disent à un linguiste de passage que je sais que l'étymologie est débattue, et ce sont des invitations à argumenter — un engagement que je n'ai pas à fabriquer.

JAMAIS OBLIGATOIRE, JAMAIS BLOQUANT

L'étymologie ne déclenche aucune tâche, ne compte dans aucune métrique de complétude, et une entrée de lexique sans ligne d'origine n'est pas incomplète. Quelqu'un sait comment sa grand-mère dit un mot ; il ne sait généralement pas que ce mot vient du turc ottoman, et le demander produirait soit des suppositions, soit l'arrêt pur et simple de sa contribution. C'est un savoir en opt-in, de la part de qui l'a — donc pas de champ origine sur le formulaire de contribution. Ça vit sur la page de l'entrée de lexique comme « ajoutez ce que vous savez », jamais sur le chemin entre l'arrivée et la soumission (§9.6).

Un piège juridique : une dérivation recopiée d'un dictionnaire étymologique publié est le travail savant de quelqu'un d'autre, et le palier d'export `etymology` (§19.2) en fait une question de redistribution. Donc les origines sont soit du savoir de contributeur, soit citées, et note porte la citation quand il y en a une.

7.7 Traductions — clés explicites, pas de polymorphisme

Une phrase peut avoir plusieurs traductions valides, donc les traductions sont des lignes. Mais *comment* elles pointent vers leur cible compte autant que le fait qu'elles existent.

La forme tentante est `target_type` plus un `target_id` `uuid` brut. Postgres ne peut absolument rien contraindre là-dessus : pas de clé étrangère, pas de comportement de suppression, aucune garantie que l'id pointe vers quelque chose de réel.

Supprimer une variante et ses traductions deviennent des orphelines *que rien dans la base ne peut détecter*. Elles s'accumulent en silence et finissent par se retrouver dans un export.

```
translations
  id uuid pk
  corpus_item_id      uuid REFERENCES corpus_items(id) ON DELETE CASCADE
  lexicon_entry_id    uuid REFERENCES lexicon_entries(id) ON DELETE CASCADE
  lexicon_variant_id  uuid REFERENCES lexicon_variants(id) ON DELETE CASCADE
  target_lang enum, text, source enum, translator_id fk
  is_preferred bool, quality_score numeric(3,2)

CONSTRAINT single_translation_target CHECK (
  (corpus_item_id IS NOT NULL)::int + (lexicon_entry_id IS NOT NULL)::int
  + (lexicon_variant_id IS NOT NULL)::int = 1 )

index (corpus_item_id, target_lang) where corpus_item_id is not null
index (lexicon_entry_id, target_lang) where lexicon_entry_id is not null
index (lexicon_variant_id, target_lang) where lexicon_variant_id is not null

-- une préférée par langue, par type de cible
unique (corpus_item_id, target_lang) where is_preferred and corpus_item_id is not null
unique (lexicon_entry_id, target_lang) where is_preferred and lexicon_entry_id is not null
unique (lexicon_variant_id, target_lang) where is_preferred and lexicon_variant_id is not null
```

Un peu plus verbeux que la version polymorphe, et ça vaut le coup — l'intégrité devient le travail de la base de données au lieu d'être quelque chose que l'appli est censée se rappeler à 1h du matin. Les trois index « préférée » sont nécessaires, pas un seul : le lexique est exactement l'endroit où une double « glose française préférée » ferait mal, parce que le traducteur la lit directement.

Le seul endroit où je garde du polymorphisme, exprès. *corrections* (§7.8) utilise encore `target_table` + `target_id`, parce qu'une correction peut cibler presque n'importe quel champ de presque n'importe quelle table, et trois colonnes nullable deviendraient quinze. Compromis accepté : ça ne peut pas cascader et c'est validé côté code applicatif. Ça tient parce que les corrections sont un enregistrement de workflow, pas du contenu de corpus, et parce que rien n'est jamais supprimé pour de vrai (§4.3).

7.8 Tables de support

<code>sources</code>	<code>id, kind (youtube forum book subtitle contribution elicitation wikipedia commoncrawl tatoeba), name, url, license_default, active, last_run_at, rate_limit, proxy_pool, notes</code>
<code>submission_meta</code>	<code>id, corpus_item_id, contributor_name, contributor_email (nullable), self_reported_region, self_reported_origin, session_id, ip_hash, consent_given, consent_voice</code> -- les métadonnées d'un élément de corpus contribué, nommée à part de -- « contribution » l'acte, pour que les deux ne partagent pas un mot
<code>corrections</code>	<code>id, target_table, target_id, field, old_value, new_value, reason, proposed_by, status, reviewed_by</code>
<code>flags</code>	<code>id, target_type, target_id, reason enum, note, session_id, ip_hash, status enum, resolved_by</code> -- reason : offensive personal_data wrong copyright other -- personal_data réutilise la routine anonymise_revoked (§4.3) une fois -- qu'un admin confirme – le même nettoyage, déclenché par un signalement -- tiers plutôt qu'une auto-révocation, pas un chemin manuel séparé
<code>users</code>	<code>id, role, display_name, email, password_hash, trust_level, region_self_reported, active</code>
<code>audit_log</code>	<code>id, actor_id, action, entity_type, entity_id, diff jsonb, ip_hash</code>
<code>stored_objects</code>	<code>id, kind (voice_clip dataset_snapshot db_backup book_scan other), bucket, storage_key UNIQUE, mime_type, size_bytes, checksum, status (pending stored deleted), is_public, expires_at, uploaded_by</code> -- source de vérité unique pour tout ce qui est en stockage objet, -- au lieu d'une colonne storage_key brute sur chaque table qui en a besoin
<code>dataset_snapshots</code>	<code>id, version, corpus_item_count, stored_object_id fk UNIQUE, rule_version, notes</code>
<code>model_versions</code>	<code>id, snapshot_id fk, metrics jsonb, trained_at, is_production</code>
<code>standardisation_rules</code>	<code>id, rule_version, pattern, replacement, script, description, active</code>

Chaque table porte aussi `created_at` et `updated_at`, avec fuseau horaire. Je n'ai laissé `updated_at` hors des blocs ci-dessus que pour qu'ils restent lisibles — ce n'est optionnel nulle part.

7.9 Des champs bon marché maintenant, impossibles plus tard

CHAMP	POURQUOI IL DOIT ÊTRE LÀ DÈS LA PREMIÈRE LIGNE
<code>license</code> <code>rights_status</code>	Le contenu scrapé et le contenu contribué portent des droits différents. Sans ça, il est impossible d'isoler un jour un sous-ensemble propre commercialement.
<code>dataset_split</code>	Fixé à l'insertion, jamais modifié. Sans ça, impossible de faire une revendication de précision honnête — ce serait corriger ses propres copies.
<code>rule_version</code>	Les règles orthographiques changeront. Les instantanés déjà publiés doivent rester reproductibles après ça.
<code>is_machine</code>	Sépare la supposition d'une machine du jugement d'un humain. Sans ça, la sortie d'un modèle contamine les données d'entraînement et personne ne le remarque.
<code>version</code>	Permet au moteur de tâches de distinguer « encore valide » de « à refaire » (§8.1).
<code>consent_voice</code>	Le consentement texte ne couvre pas la voix, et on ne peut pas le recueillir rétroactivement (§4.2).

7.10 Publications — phase 2

Une page qui rassemble les travaux sur le derja tunisien — articles, thèses, corpus, outils — que les gens peuvent enrichir et discuter.

```
publications      id, title, authors, year, venue, doi, url,
                  kind enum (paper|thesis|corpus|tool|book),
                  abstract, submitted_by, status enum
                  unique (doi) where doi is not null

publication_comments id, publication_id fk, body, author_name,
                    session_id,          -- même modèle anonyme que les contributions
                    status enum
```

LES COMMENTAIRES SONT UNE SURFACE DE MODÉRATION QUE RIEN D'AUTRE ICI NE CRÉE

Tout le reste dans ce système est borné — un tag de région, une traduction, un vote. La discussion en texte libre ne l'est pas, et c'est la seule partie qui peut générer du travail qui n'est pas du travail de corpus. Donc : même modèle de session que les contributions, tout ce qui est en dessous du niveau de confiance 1 retenu pour approbation, et **la fonctionnalité est carrément désactivée si elle coûte plus d'attention qu'elle n'en rapporte**. L'écrire maintenant fait que la désactiver plus tard est une décision toujours sur la table, pas un échec.

L'amorcer avant de le lancer — une bibliographie vide a l'air abandonnée. La préremplir avec les travaux cités au §1.2 et au §13.1 : CODA-TUN, NOTA, Tunisiya, TARIC, TSAC, TEC, TunSwitch, TEDxTN, MADAR, MyVoice, DARIJA-C, KALAM'DZ. Cette lecture est déjà faite ; la page, c'est surtout de la saisie.

Un chercheur qui arrive ici devrait trouver le domaine, pas seulement ce projet. Être l'endroit où se rassemblent les travaux sur le derja tunisien est une façon moins chère de devenir une référence que de produire le meilleur corpus — et ça transforme le §1.2 d'un paragraphe défensif en une page vivante. C'est aussi la voie la plus directe vers le problème 4 du §5 : les gens qui utilisent une bibliographie de TAL tunisien sont des gens qui pourraient relire des données du nord-ouest ou du sud.

7.11 Média — phase 5, vide pour l'instant

Ces tables existent dans le schéma et ne contiennent rien. Elles sont là parce que la question de consentement qui leur est rattachée ne peut pas être répondue rétroactivement (§4.2).

```
speakers      id, session_id, region_origin enum, region_residence enum,
              age_band enum, consent_voice bool

media         id, lexicon_variant_id fk, lexicon_form_id fk null, speaker_id fk,
              stored_object_id fk UNIQUE,  -- stockage objet, pas Postgres (§7.8)
              duration_ms, sample_rate, consent_scope enum
```

Le consentement voix apparaît trois fois dans ce schéma, exprès, pas par accident : `submission_meta.consent_voice` (§7.8) est l'opt-in capturé au moment de la soumission, avant même qu'un enregistrement n'existe. `speakers.consent_voice` est ce même fait recopié en avant sur un profil de locuteur une fois qu'il en existe un en phase 5. `media.consent_scope` est celui qui gouverne vraiment un extrait précis — texte seulement contre voix incluse — et fait autorité si les deux autres venaient à diverger, puisque c'est le plus proche de l'enregistrement réel.

Basique veut dire basique. La voix de la phase 5, c'est de l'audio d'attestation : quelqu'un enregistre un extrait d'une variante qu'il a déjà soumise, et ça se rejoue sur l'entrée de lexique. Pas de reconnaissance vocale, pas de synthèse vocale, pas d'alignement forcé, pas de modèles de parole. L'extrait prouve qu'une forme écrite se dit vraiment comme ça, par quelqu'un de là-bas — la seule chose que le texte ne peut pas prouver tout seul.

L'audio change trois choses structurellement, et c'est pour ça que ce n'est pas juste « une migration plus tard » :

1. **Le stockage quitte Postgres.** Un élément de corpus fait environ 1 Ko. Un extrait de dix secondes en fait environ 150. Ça veut dire du stockage objet avec une politique de cycle de vie, et `pg_dump` ne le couvre pas — un second chemin de sauvegarde avec sa propre économie et son propre test de restauration (§2.5). La ligne `stored_objects` se résout via une URL présignée — l'extrait lui-même ne passe jamais par l'API. Le fournisseur derrière est interchangeable (disque local en dev, R2 ou S3 en production, une seule interface, §10.0) — **construit** comme `control-plane/backend/core/src/storage/`, avec un adaptateur local et un adaptateur R2 déjà écrits, juste pas encore branchés sur une route — donc cette décision n'est pas à reprendre deux fois.
2. **speakers n'a pas d'équivalent texte.** Un enregistrement atteste une variante *telle que parlée par une personne* dont l'origine peut ne pas correspondre à la région de la variante.
3. **Le consentement cesse d'être nettoyable.** Le §4.3 fonctionne parce qu'une phrase dont les identifiants ont été retirés est vraiment anonyme. Un enregistrement ne l'est pas — c'est quasi-biométrique et on ne peut pas l'anonymiser en mettant une colonne à null. **La révocation voix veut dire suppression**, la deuxième exception au principe « rien n'est jamais supprimé pour de vrai ».

Le moteur de tâches

La pièce que je regretterais le plus de rater.

Tout ce qui suit concerne la curation et la vérification de matériau qui existe déjà dans le pipeline — relecture, tagging, traduction, adjudication. Ce n'est pas la seule porte d'entrée. N'importe qui peut ajouter du matériau tout neuf directement, sans tâche requise : un nouveau mot, une phrase ou un paragraphe sur `/contribute`, une nouvelle origine de mot sur `/origins`, une nouvelle citation sur `/research`. Cette section est la couche de relecture posée par-dessus, pas un portail devant.

Contribuer, ce n'est pas seulement écrire quelque chose et repartir, non plus. Celui qui écrit un élément de corpus peut, en option, le traduire lui-même, ajouter une graphie arabe ou arabizi, et dire à quelle région ça appartient — derja pan-tunisien standard, ou une région précise s'il la connaît. Tout ce qu'il saute ne se perd pas et ne reste pas à moitié fait : ça devient exactement la même tâche `translate_*`, `transliterate_*` ou `region_tag` (§8.2) qu'un élément de corpus scrapé aurait déclenchée. Écrire son propre matériau et compléter celui de quelqu'un d'autre, c'est la même file de tâches, juste abordée par les deux bouts.

UNE SEULE COLONNE DE STATUT NE PEUT PAS FAIRE ÇA

Une entrée peut être traduite mais pas translittérée, ou translittérée mais pas relue. Des axes de complétude séparés. Une seule colonne de statut les force dans un ordre qui n'existe pas et rend invisible le travail à moitié fait.

8.1 Schéma

```
tasks
  id, corpus_item_id fk, type enum
  status      enum    -- open | claimed | done | rejected | needs_rework
  priority    int
  claimed_by  fk null, claimed_at, completed_by fk null, completed_at
  output_record_id uuid    -- pointe vers la ligne qu'elle a produite
  requires_role enum
  target_regions enum[] null -- les régions en conflit, ou l'indice (§8.5)
  item_version int
  idempotency_slot text not null default ''
  idem_key    text    -- sha256(corpus_item_id : type : item_version : idempotency_slot)

  index (status, type, priority desc)
  index (claimed_by, status)
  index (corpus_item_id)

  -- au plus UNE tâche vivante par (élément de corpus, type, version, slot)
  unique (idem_key) where status in ('open','claimed','needs_rework')
```

POURQUOI LA CLÉ D'IDEMPOTENCE EXISTE

« Ouvrir la tâche Y si elle n'est pas déjà là » a l'air très bien jusqu'à ce que quelqu'un corrige un élément de corpus déjà traité. La correction arrive, l'élément change, les règles de génération se déclenchent à nouveau et remettent en file `translate_msa` et `link_lemma` — les deux déjà terminées. Affiner une autre annotation, même chose. La file se remplit de doublons de travail déjà clos et les relecteurs arrêtent de faire confiance à leur boîte de réception, qui est la seule surface par laquelle passe toute la boucle de contribution.

- **corpus_items.version** monte à chaque changement accepté — une correction appliquée, une standardisation, un tag arbitré. C'est ce que veut dire « cette tâche porte sur cet état ».

- **idem_key** vaut `sha256(corpus_item_id : type : item_version : idempotency_slot)`. Mêmes quatre entrées, même clé, doublon rejeté par l'index au lieu d'être mis en file.
- **idempotency_slot sépare plusieurs tâches légitimes du même type sur un même élément de corpus.** Vide pour le travail à l'échelle de l'élément (`review`, `region_tag`, `translate_*`, `transliterate_*`, `standardise`), `char_start:char_end` pour le travail à l'échelle d'un intervalle (`link_lemma`, `adjudicate`), un ordinal pour le cas N-parmi-plusieurs (`confirm` → 1, 2). L'omettre et l'index unique fusionne discrètement deux confirmations en une, un seul lien de lemme par phrase au lieu d'un par token, et un second intervalle disputé dans le néant. Rien de tout ça ne lève d'erreur.
- **L'index est partiel, sur les statuts vivants seulement.** Mettre `status` dans la clé unique elle-même a l'air équivalent et ne l'est pas — ça contraint aussi les lignes historiques et bloque les transitions de reprise légitimes. Le limiter à `open/claimed/needs_rework` dit l'intention réelle : jamais deux tâches vivantes pour le même travail, laisser l'historique clos tranquille.
- **Un incrément de version est ce qui rouvre vraiment le travail**, et tous les incréments n'invalident pas tout. Chaque règle déclare quels changements tuent sa sortie — un nouveau tag de région n'invalidé pas une translittération terminée ; une standardisation invalide bien `link_lemma`. Cette correspondance vit comme donnée dans le module de règles.

Construit. Le schéma, la réclamation atomique, le cycle de vie complet/rejet/reprise, la clé d'idempotence exactement comme ci-dessus, et les règles de génération pour chaque ligne du §8.3 — tout ça vit dans `control-plane/backend/core/src/tasks/`, un paquet sans dépendance à un framework que la couche NestJS appelle. Les complétions `review`, `region_tag` et `confirm` sont branchées de bout en bout et ont été testées contre une base de données en fonctionnement. Les complétions `translate_*`, `transliterate_*`, `standardise`, `adjudicate` et `link_lemma` ne sont pas encore branchées — elles ont besoin des modules lexique et traduction, qui ne sont encore que des coquilles vides, et `link_lemma` a en plus besoin d'une passe de tokenisation qui n'existe pas encore, puisque rien n'écrit actuellement de lignes `tokens`.

8.2 Types de tâches

TYPE	PRODUIT	SLOT	RÔLE MIN.
<code>review</code>	tag qualité + région, ou un rejet	vide	reviewer
<code>region_tag</code>	tag de région	vide	contributor
<code>confirm</code>	vote d'accord sur un tag existant	ordinal	contributor
<code>translate_msa / _fr / _en</code>	ligne de traduction	vide	contributor
<code>transliterate_to_arabic</code>	ligne de forme de lexique, script = arabic	vide	contributor
<code>transliterate_to_arabizi</code>	ligne de forme de lexique, script = arabizi	vide	contributor
<code>standardise</code>	<code>canonical_form</code> + <code>canonical_map</code>	vide	reviewer
<code>adjudicate</code>	la décision finale là où les annotateurs étaient en désaccord	intervalle	reviewer
<code>link_lemma</code>	une ligne <code>links</code> , token vers variante de lexique	intervalle	reviewer

8.3 Règles de génération

```

si élément de corpus ingéré      -> générer review
si review approuvée              -> générer region_tag, translate_msa
si region_tag terminée           -> générer confirm slot=1, slot=2 (annotateurs différents)
si traduction ajoutée            -> si arabizi : générer transliterate_to_arabic
                                -> si arabic : générer transliterate_to_arabizi
si accord du consensus < 0.6     -> générer adjudicate slot=<intervalle> (routé, §8.5)
si élément de corpus standardisé -> générer link_lemma slot=<intervalle> par token
si correction acceptée           -> incrémenter corpus_item.version
                                -> régénérer seulement ce que ce changement invalide

```

Construit, pas encore branché. Chaque règle ci-dessus existe aujourd'hui comme une fonction pure dans `control-plane/backend/core/src/tasks/spawn-rules.ts`. `si review approuvée`, `si region_tag terminée`, et la règle d'adjudication déclenchée par le consensus sont vraiment appelées depuis le code de complétion de tâches en fonctionnement. `si traduction ajoutée` et `si élément de corpus standardisé` sont écrites et testées mais rien ne les appelle encore, parce que rien ne crée encore les enregistrements de traduction ou de standardisation qui les déclencheraient.

8.4 Réclamation et libération

- La réclamation est atomique — `UPDATE ... WHERE status='open' RETURNING`. Deux workers n'obtiennent jamais la même ligne.
- Les réclamations expirent ; une tâche libère tout ce qui est détenu depuis plus de deux heures.
- Un worker détient au plus 5 réclamations à la fois, pour que personne n'accapare la file.
- `needs_rework` n'est pas `rejected` : la reprise retourne dans la file, le rejet la ferme.

8.5 Routage régional, et le biais qu'il empêche

L'ÉCHEC CONTRE LEQUEL JE CONÇOIS CE SYSTÈME

Quand le consensus tombe sous le seuil, une tâche `adjudicate` se déclenche. Si le bassin de relecteurs est très nordiste — et au début il le sera, parce que c'est là qu'est mon réseau — un relecteur du nord peut rejeter en toute confiance une variante du sud parfaitement bonne juste parce qu'il ne l'a jamais entendue. Ça ne fait pas que perdre une ligne. Ça élimine systématiquement exactement le matériau que ce projet existe pour collecter, tout en ayant l'air d'être du contrôle qualité pendant tout ce temps.

Donc l'adjudication ne va pas à qui est disponible. La tâche porte `target_regions` — un ensemble, parce que le §6.4 a fait de la région un ensemble et qu'un conflit nord-ouest contre sud contient deux régions.

```

quand adjudicate est généré :
  target_regions = les régions des tags en conflit, ou le region_hint du document de l'élément

1. proposer UNIQUEMENT aux relecteurs dont la région auto-déclarée est DANS target_regions
2. après 7 jours non réclamée -> ouvrir à tout le monde, marquer out_of_region, et le dire :
   « aucun relecteur de cette région n'était disponible »
3. une adjudication out_of_region peut confirmer ou remonter à un admin.
   Ne jamais rejeter pour des raisons régionales, ne jamais supprimer une variante.

```

Un relecteur correspondant à n'importe quelle région de l'ensemble compte comme étant de la région. La fenêtre de sept jours est par tâche, pas par région — sinon un conflit à deux régions attendrait deux fois plus longtemps qu'un conflit à une région, ce qui est à l'envers, puisque les cas disputés sont ceux qui ont le plus besoin d'être tranchés.

- Même idée pour `region_tag` et `confirm` : une confirmation d'un relecteur de cette région compte à plein poids, une venue de l'extérieur compte moins.

- Le taux `out_of_region` est sur l'aperçu admin (§17.1). S'il grimpe, la réponse est de recruter dans cette région — problème 4 du §5 — pas de relâcher le seuil.
- C'est pour ça que `region_self_reported` est sur la table `users` et est demandé à l'invitation d'un relecteur, plutôt que d'être une décoration de profil optionnelle.

8.6 Ce que ça permet de répondre facilement

Ce qui n'est pas fini, par type et par âge. Quelle région a le plus de travail bloqué. Qui contribue quoi et à quelle vitesse. Si le pipeline est engorgé à la relecture, à la traduction ou à la standardisation. Des requêtes, pas des enquêtes.

Comptes et permissions

La friction tue le volume. L'anonymat tue la qualité. On choisit par rôle, pas une fois pour toutes.

9.1 La tension

Chaque étape entre « je veux aider » et « j'ai soumis quelque chose » coûte des contributions, et les contributions sont la ressource sur laquelle tourne tout ce projet. Mais la qualité de l'annotation a besoin de responsabilisation. Une seule politique ne peut pas servir les deux, donc il n'y en a pas qu'une.

POURQUOI LES CONTRIBUTEURS N'ONT PAS DE VRAIS COMPTES

Il existe quelque part une version de ce projet où les contributeurs s'inscrivent proprement — email, mot de passe, vérification, un profil. C'est plus propre. Chaque règle devient plus simple. Je saurais exactement qui a écrit quoi.

C'est aussi un fantasme, parce que personne ne fait ça. Imaginez la scène : quelqu'un clique sur un lien, il est curieux, il a peut-être quinze secondes de bonne volonté, et la première chose qu'il rencontre, c'est *Créez votre compte*. Parti. Pas agacé, pas en train de peser le pour et le contre — parti, instantanément, comme je le serais moi-même. Je demanderais à un inconnu d'ouvrir un compte sur un site dont il n'a jamais entendu parler, tenu par un type dont il n'a jamais entendu parler, pour qu'il puisse offrir un mot gratuitement.

Donc les contributeurs ont un cookie et une case pour leur nom, et je vis avec des garanties d'identité plus faibles. C'est tout le compromis, et ce n'est pas serré.

9.2 Niveaux

RÔLE	AUTH	PEUT FAIRE
anonymous	aucune	naviguer, chercher, télécharger, voir les statistiques
contributor	nom + email optionnel, pas de mot de passe	soumettre, confirmer et voter, proposer des corrections, tâches simples
trusted contributor	email vérifié, confiance ≥ 2	+ traduction, translittération
reviewer	authentification complète, sur invitation seulement	+ file de relecture, standardisation, adjudication
admin	authentification complète + 2FA	+ utilisateurs, sources, publication d'instantanés
superadmin	authentification complète + 2FA	tout, y compris les opérations destructrices

9.3 Comment fonctionne la contribution anonyme

```

session_id // cookie httpOnly, uuid, 1 an – relie entre elles les soumissions d'une même personne
ip_hash    // sha256(ip + sel serveur) – détection d'abus seulement, jamais une IP brute
name       // texte libre, affichage seulement, PAS une revendication d'identité
email      // optionnel, pour le crédit et le suivi
consent    // case à cocher explicite, stockée par soumission

```

Intégrité du classement : classé par contributions *approuvées*, jamais par soumissions brutes. Classer par brut reviendrait à payer les gens pour spammer. Les collisions de noms ne posent pas de problème — c'est de la motivation, pas un justificatif. C'est l'identifiant de session qui agrège vraiment le travail de quelqu'un.

9.4 Niveaux de confiance

confiance 0	nouveau / anonyme	-> confirmer + soumettre seulement
confiance 1	3+ entrées approuvées	-> tâches region_tag
confiance 2	email vérifié, 10+ approuvées, <10 % rejetées	-> traduction, translittération
confiance 3	10+ approuvées sur 2+ types de tâches, <5 % rejetées	-> éligible à une invitation de relecteur

Calculé chaque nuit à partir de l'historique d'approbation, pas distribué à la main. La confiance se gagne ; le rôle de relecteur se donne. L'éligibilité ne promeut jamais automatiquement — un admin envoie quand même l'invitation, et capture la région du relecteur en le faisant, parce que le routage en dépend (§8.5).

9.5 Connexion des relecteurs

Un vrai compte, pas un cookie — email et mot de passe, ou GitHub OAuth. Chaque action privilégiée écrit dans `audit_log` avec acteur, diff et `ip_hash`. Le design des tokens, des sessions et de la 2FA derrière tout ça est un sujet à part entière, couvert intégralement aux §9.7–§9.10 plutôt que répété ici.

Construit jusqu'ici : email/mot de passe seulement — hachage `argon2id`, JWT access + refresh avec rotation et détection de réutilisation, double-soumission CSRF, la chaîne complète `ThrottlerGuard` → `AuthGuard` → `RolesGuard` → `TrustGuard` (§9.6, §9.10). GitHub OAuth n'est pas encore branché. L'inscription a brièvement eu un vrai bug qui mérite d'être noté plutôt que corrigé discrètement et oublié : chaque inscription était codée en dur sur `role: 'reviewer'`, donc quiconque s'inscrivait obtenait un accès relecteur. Trouvé et corrigé avant que ça ne quitte jamais une base de dev — ça part maintenant par défaut sur `contributor` comme il se doit — mais c'est le genre d'erreur d'une ligne qui mérite d'être nommée pour que ça n'arrive pas deux fois.

9.6 La hiérarchie reste interne

Niveaux de confiance, rôles, types de tâches, règles de génération, seuils de consensus, routage d'adjudication — tout ça, c'est ce qui garde le système honnête. Rien de tout ça n'est le problème d'un contributeur et rien de tout ça n'apparaît devant lui.

Un contributeur voit une case, une question, un bouton. Il ne voit pas son niveau de confiance, ne sait pas que finir un tag de région a déclenché deux tâches de confirmation, ne choisit pas de type de tâche, et ne se fait jamais dire qu'il n'est éligible à rien. Le travail qu'il ne peut pas encore faire n'est simplement pas proposé — pas d'état verrouillé, pas de barre de progression vers une permission, pas d'explication d'un système qu'il n'a rien demandé à connaître. La promotion se fait en silence : plus de types de travail commencent à apparaître.

Le classement est le seul statut visible où que ce soit, et c'est un classement plutôt qu'un rang — qui a contribué, pas à quel palier tout le monde se situe.

LA RÈGLE POUR CHAQUE ÉCRAN DESTINÉ AUX CONTRIBUTEURS

Une question, trois secondes, aucune explication requise. Si un écran a besoin d'un paragraphe d'instructions, c'est l'écran qui a tort, pas la personne qui le lit.

Chaque phrase entre l'arrivée et la soumission est prise sur un budget compté en secondes. Je n'ai pas de citation pour dire combien exactement et je n'en ai pas besoin — je sais à quelle vitesse je ferme un onglet qui veut que je lise quelque chose avant de me laisser faire quoi que ce soit, et je n'ai aucune raison de penser que qui que ce soit sera plus patient avec le mien que je ne le suis avec le leur.

Toute cette complexité reste entièrement instrumentée. Elle fait juste face au tableau de bord admin (§17.1) au lieu du contributeur.

9.7 Design des tokens et des sessions

JOSE (HS256), pas une bibliothèque JWT au hasard — une implémentation bien audité plutôt que choisir le paquet qui a le plus d'étoiles cette année. Un token porte quatre choses en plus du sub et de l'expiration habituels : `token_version` (incrémenté pour invalider tous les tokens d'un utilisateur d'un coup — le bouton « déconnecter partout » et « quelque chose ne va pas, tue les sessions de ce compte » réunis dans un seul champ), `sid` (identifiant de session, pour qu'une connexion puisse être révoquée sans toucher aux autres), `rjti` (un identifiant de token tournant sur le refresh token spécifiquement, pour qu'un refresh token volé et utilisé deux fois soit détectable — la réutilisation veut dire que quelqu'un d'autre l'a). Les mots de passe sont en `argon2id`, réservés aux relecteurs et admins puisque les contributeurs n'ont pas du tout de mot de passe (§9.3).

9.8 Flux de session

Token d'accès court, refresh token plus long, paire standard. Le refresh fait tourner le token à chaque utilisation — l'ancien `rjti` arrête de fonctionner dès qu'un nouveau est émis, ce qui est ce qui rend la détection de réutilisation possible. La déconnexion révoque le `sid` côté serveur plutôt que de simplement supprimer un cookie côté client, pour qu'un token volé ne reste pas valide après que l'utilisateur légitime pense s'être déconnecté. Incrémenter `token_version` est l'instrument brutal pour « tout, partout, maintenant » — utilisé pour un changement de mot de passe ou une compromission suspectée, pas pour une déconnexion de routine.

9.9 Vérification d'email et réinitialisation de mot de passe

Les deux passent par un OTP à usage unique et à durée limitée envoyé par email, pas un lien magique avec un token longue durée qui traîne indéfiniment dans une boîte de réception. Limité en débit par adresse et par IP, et la réponse est identique que l'adresse existe ou non dans le système — « si un compte existe, on a envoyé un code » — pour que l'endpoint ne puisse pas servir à vérifier qui a un compte ici.

9.10 Spécificités de la connexion relecteur et admin

- Les connexions ratées verrouillent temporairement le compte après une poignée de tentatives, avec un backoff, pas un verrouillage permanent et dur pour lequel un utilisateur légitime devrait ensuite m'écrire un email.
- La 2FA TOTP est obligatoire pour admin et superadmin, optionnelle mais encouragée pour les relecteurs.
- GitHub OAuth est préféré au mot de passe pour les relecteurs spécifiquement parce que la plupart d'entre eux sont probablement des gens techniques qui ont déjà un compte GitHub — un mot de passe de moins dont je suis responsable.

DÉLIBÉRÉMENT PAS REPRIS D'AILLEURS

Quelques motifs qui apparaissent dans d'autres systèmes que j'ai construits ou vus, et qui n'ont pas leur place ici, dits explicitement pour que personne ne se demande pourquoi ils manquent : **l'usurpation d'identité** (un admin qui se connecte « en tant que » un utilisateur) — rien ici n'en a besoin et c'est une responsabilité qui traînerait sans être utilisée. **Le chiffrement de numéro d'identité nationale / CIN** — ce projet ne collecte jamais d'identifiant gouvernemental, donc rien à chiffrer. **Les paiements** — déjà couvert (§4), il n'y a pas de palier payant, donc pas de flux de paiement à sécuriser. **Un flux de token mobile séparé** — il n'y a pas d'appli native (§20), donc pas de client séparé pour lequel concevoir des tokens.

9.11 Envoi d'emails

Motif outbox : écrire « envoyer cet email » est une ligne de base de données dans la même transaction que ce qui l'a déclenchée, pas un appel direct à une API d'email au milieu d'une requête. Une tâche de fond dispatche l'outbox de façon asynchrone et réessaie en cas d'échec. Ça veut dire qu'un fournisseur d'email capricieux ne peut jamais faire échouer l'action réelle — une inscription réussit même si l'email de vérification est retardé de quelques minutes, au lieu que toute la requête échoue parce que Resend a eu une mauvaise seconde. Resend est le principal, avec SMTP comme chemin de repli plutôt qu'une deuxième chose à configurer et à oublier (§2.6).

10

Architecture backend

Un monorepo, deux applications. Toujours pas de ML dans le chemin de requête — ça n'a pas changé.

10.0 Stack technique

Le tableau complet avant le raisonnement. Tout ce qui suit est une décision, pas une case en attente — si ce document dit « à définir » ailleurs (hébergement, domaine), c'est l'exception, pas ce tableau.

COUCHE	CHOIX	REMARQUE
Frontend	Next.js (App Router)	SSR pour les pages publiques — les exigences de citation et de recherche du §16 en ont besoin
Backend	NestJS 11, TypeScript	Possède la frontière de l'API ; la chaîne de guards remplace l'ancienne conception à app unique avec gestionnaires de routes
ORM	Prisma	Schéma dans db/
Base de données	PostgreSQL, derrière pgBouncer	Mode transaction, pool de 20 (§10.2)
Cache / sessions	Redis	Pas la file de tâches — c'est toujours pg-boss (§10.3)
File de tâches	pg-boss sur Postgres	BullMQ sur Redis est une évolution future documentée, pas le choix par défaut
Auth	jose (JOSE), HS256, argon2id	Design complet aux §9.7–§9.11
Service NLP	Python, FastAPI	Health-check seulement, phase 1 — construit , conteneurisé ; le vrai travail de modèle est en phase 4, hors machine (§11)
Service Processing	Rust, Actix	Health-check seulement — construit , conteneurisé ; le reste tant que le profilage du pipeline TypeScript à l'échelle réelle du corpus n'a pas montré qu'une étape précise (dédoublonnage MinHash/LSH, tokenisation, export) est le goulot d'étranglement — pas une simple échéance de phase (§7.11)
Reverse proxy	Caddy (interne)	
Edge / tunnel	Cloudflare Tunnel (cloudflared)	Aucun port entrant ouvert sur le VPS
Email	Resend, repli SMTP	Motif outbox (§9.11)
Système de design	shadcn/ui + Tailwind	Écarts RTL vérifiés, pas supposés résolus (§15.2)
Gestionnaire de processus	pm2	Fait tourner le conteneur NestJS en mode cluster, 2 instances, seuil de redémarrage par instance — construit , <code>control-plane/backend/ecosystem.config.js</code>
Suivi d'erreurs	Sentry (@sentry/node, @sentry/react)	Gratuit à cette échelle — c'est comme ça que le « taux d'erreur » de l'aperçu admin (§17.1) aura une vraie source au lieu d'être un chiffre sans rien derrière. Pas encore branché dans le backend.
Observabilité	OpenTelemetry + logs structurés	Une machine gérée en solo sans logs structurés est indéboguable au moment où quelque chose casse sans personne autour pour demander. Pas encore branché.
Analytics	PostHog (palier gratuit)	Remplace une hypothèse antérieure d'un Umami/Plausible auto-hébergé (§20) — réutiliser un outil déjà connu bat réévaluer les plateformes d'analytics depuis zéro. Pas encore branché.
Stockage objet	Interchangeable : disque local (dev) R2 S3	Une interface, fournisseur changé par variable d'environnement. Uploads et téléchargements passent par des URLs présignées — les fichiers ne traversent jamais l'API (§7.11). Construit comme interface plus les deux adaptateurs (<code>control-plane/backend/core/src/storage/</code>) ; pas encore appelé depuis une route.
Temps réel	Pas par défaut — sondage	socket.io + un adaptateur Redis est le chemin de mise à niveau si l'aperçu admin ou la file de relecture ont vraiment besoin de push plutôt que de sondage un jour (§17.1). Pas coupé par réflexe ; juste pas encore nécessaire.

COUCHE	CHOIX	REMARQUE
CI/CD	GitHub Actions	master.yml (lint/typecheck/test/dérive de migration/CodeQL, construit) et verify-backup-restore.yml (§2.3, construit) — infra-watch.yml pas encore construit
Gestionnaire de paquets	pnpm, un workspace, un lockfile	

```

open_derja/      -- UN dépôt, UN workspace pnpm, UN lockfile
├── control-plane/ -- tout ce qui est TypeScript : E/S, orchestration, machines à états
│   ├── backend/  -- NestJS : chaîne de guards, logique métier, accès aux données
│   │   └── core/  -- logique de domaine : tâches, consensus, confiance, standardisation,
│   │               stockage. Sans framework, son propre paquet workspace,
│   │               importé par backend, worker et scrapers
│   ├── worker/   -- tâches planifiées pg-boss ; expire_claims en fonctionnement,
│   │               dispatcher d'outbox pas encore construit
│   └── scrapers/  -- runners d'ingestion (amorcé comme paquet,
│                   pipeline/ et sources/ encore vides)
├── data-plane/    -- Rust, Actix – travail par lots déterministe, gourmand en CPU.
│                   Health-check seulement jusqu'à ce qu'une étape se révèle être
│                   un goulot d'étranglement à l'échelle réelle du corpus
├── intelligence-plane/ -- Python, FastAPI – health-check seulement, phase 1
│   └── nlp/
├── frontend/      -- Next.js : site public + tableaux de bord
├── shared/         -- types, enums, schémas zod. Zéro dépendance, zéro logique.
├── db/             -- schéma prisma, migrations, seeds
├── devops/         -- Dockerfiles, compose, Caddy/cloudflared, sauvegardes
├── docs/           -- ce grand livre, le blueprint, VOICE.md, CONTRIBUTORS.md,
│                   journal de vérification
└── testing/        -- e2e, organisé par surface

```

Les trois groupes de premier niveau — `control-plane`, `data-plane`, `intelligence-plane` — correspondent à ce qui varie vraiment dans la façon dont chaque brique tourne : TypeScript pour tout ce qui dépend d'E/S ou d'une machine à états, Rust pour le travail par lots gourmand en CPU une fois qu'il s'avère nécessaire plutôt que supposé, Python pour l'inférence et l'entraînement tenus à l'écart du chemin de requête. `frontend`, `shared`, `db` et `devops` restent hors de ce découpage exprès — ils ne sont pas propres à une plateforme. `shared` en particulier est le vocabulaire que chaque plateforme doit partager, et comme rien d'écrit en Rust ou en Python ne peut importer un paquet TypeScript de toute façon, le nichier dans `control-plane` dénaturerait ce qu'il est vraiment. `core` vit sous `control-plane/backend/` plutôt qu'à la racine de `control-plane/` — ça reste son propre paquet workspace sans framework, juste imbriqué un niveau sous l'application qui s'en sert le plus. La direction des dépendances n'a pas changé : `backend` → `core` → `db/shared`, `frontend` ne parle jamais qu'à `backend` via le client typé, rien n'importe vers le haut. C'est un seul monorepo de bout en bout, pas cinq dépôts séparés — toute la raison d'être de `shared`, c'est qu'une région, un rôle ou un type de tâche ne puisse pas signifier deux choses différentes à deux endroits différents, et ça ne tient que si tout est dans un seul workspace avec un seul lockfile.

nlp/ est en Python et ne peut pas importer shared directement. Ce n'est pas un problème pour l'instant : ni nlp/ ni le service Rust ne portent de logique sensible aux enums dans leur forme phase-1 health-check-seulement — rien là-dedans ne décide une région, un rôle, ou un type de tâche. À revoir avec un contrat OpenAPI/JSON généré depuis shared seulement si un service Python commence à prendre des décisions qui doivent s'accorder avec ce vocabulaire — le plus vraisemblablement quand le classificateur de dialecte de la phase 4 arrivera.

10.1 Forme

Deux applications, pas une, et toujours un seul dépôt. NestJS possède la frontière de l'API et la chaîne de guards — authentification, validation, logique métier, accès aux données. Next.js possède le rendu — le site public et les tableaux de

bord — et ne parle à NestJS que via un client typé, jamais directement à la base de données. L'arborescence des fichiers est au §10.0 ; rien ci-dessous ne la redérive.

Séparer l'application n'a pas changé pourquoi c'est un seul dépôt. Les enums — région, type de tâche, rôle, script, origine — sont définis une fois dans `shared` et importés des deux côtés, donc une liste déroulante et une contrainte de base de données ne peuvent pas diverger parce que c'est littéralement le même fichier. Les schémas zod valident à la frontière et typent les formulaires depuis une seule définition. Une installation, un pipeline CI, un déploiement — cette partie du raisonnement n'avait pas besoin de changer juste parce que le backend a eu son propre processus.

Pas des microservices, et puisque l'assistance automatique est reportée (§11), pas de vrai travail de ML non plus — les deux conteneurs de service du §10.0 existent pour prouver la forme du déploiement, pas pour prendre des décisions. Quand l'inférence par lots arrivera un jour, elle tournera hors machine (§11.3), donc elle ne devient jamais une troisième vraie charge de travail sur cette machine.

10.2 pgBouncer

Se place entre l'application et Postgres en mode pooling par transaction, pool de taille 20. Postgres a un plafond dur de connexions concurrentes, et NestJS, le worker et pg-boss qui ouvrent tous leurs propres connexions directement, c'est exactement comme une petite machine finit par en manquer sous une charge réelle — pas une hypothèse, un mode d'échec connu pour quiconque fait tourner Postgres derrière plus d'un processus.

Un invariant qui vaut la peine d'être dit clairement pour ne pas être redécouvert à la dure : `DATABASE_URL` pointe vers pgBouncer et c'est ce que l'application utilise en fonctionnement. `DIRECT_URL` pointe directement vers Postgres et c'est ce qu'utilisent les migrations Prisma — le pooling en mode transaction ne supporte pas les prepared statements et les fonctionnalités au niveau session dont les migrations ont besoin. Deux URLs, deux rôles, et les confondre est le genre d'erreur qui ne se manifeste qu'une fois, au pire moment.

10.3 Tâches de fond

Redis gagne sa place pour le cache et les sessions (§9.7), pas pour la file de tâches — ce sont des métiers différents, et les confier au même système voudrait dire qu'une panne de Redis emporte les deux d'un coup. pg-boss reste sur Postgres pour la file, pour la même raison qu'il a été choisi avant même que Redis ne soit dans le paysage : un service de moins à faire tourner, une chose de moins à sauvegarder, et le volume ici n'approche jamais de justifier une seconde technologie de file faisant le même travail que Redis pourrait théoriquement faire aussi.

TÂCHE	FRÉQUENCE	CE QU'ELLE FAIT
<code>expire_claims</code>	5 min	libère les tâches réclamées depuis plus de 2h
<code>refresh_consensus</code>	à l'écriture + toutes les heures	recalcule l'accord, marque pour adjudication
<code>recompute_trust</code>	nocturne	met à jour les niveaux de confiance des contributeurs
<code>refresh_stats</code>	10 min	couverture, classement, taux de succès du traducteur
<code>scrape_sources</code>	par source	récupère du nouveau matériau depuis les sources actives
<code>anonymise_revoked</code>	toutes les heures	nettoie les identifiants pour les consentements révoqués (§4.3)
<code>export_snapshot</code>	manuel / hebdomadaire	gèle, calcule le checksum, chiffre, upload
<code>backup_db</code>	quotidien	<code>pg_dump</code> , chiffre, expédie hors machine

CE QUE ÇA COÛTE, ET LE PLAFOND QUI COMPENSE

pg-boss sonde la base de données. Un worker qui martèle Postgres pour la génération de tâches, le consensus et l'export est en concurrence avec le même Postgres qui sert le site — sur une petite machine, la file peut affamer le frontend. Donc le sondage est plafonné plutôt que laissé aux valeurs par défaut : un plancher de 2s sur l'intervalle de sondage, un plafond dur sur la concurrence, des lots assez petits pour qu'aucune tâche ne reste sur une longue transaction. Tout ce qui est volumineux — export d'instantané, régénération après un changement de règle — tourne comme une seule tâche planifiée à une heure calme. Toutes les valeurs de config dans le panneau admin (§17.2), pas des constantes enterrées dans le code.

10.4 Conventions d'API

POST	/api/corpus-items/contribute	créer (contribution) – CONSTRUIT
GET	/api/corpus-items/:id	données de la page permalien – CONSTRUIT
GET	/api/corpus-items?region=&...	navigation / recherche
POST	/api/tasks/claim	réclamation atomique – CONSTRUIT
POST	/api/tasks/:id/complete	soumettre un résultat – CONSTRUIT
POST	/api/tasks/:id/review	complétion d'une tâche review – CONSTRUIT
POST	/api/tasks/:id/region-tag	complétion d'une tâche region_tag – CONSTRUIT
POST	/api/tasks/:id/confirm	complétion d'une tâche confirm – CONSTRUIT
POST	/api/tasks/:id/skip	libérer, sans pénalité
GET	/api/tasks/inbox	ce que cet utilisateur peut travailler
POST	/api/tags	ajouter un tag
POST	/api/corrections	proposer une correction
POST	/api/flags	signaler quelque chose (§16.4)
GET	/api/translate	le point d'entrée de recherche (§14)
GET	/api/stats/coverage	statistiques publiques de couverture

Le contrat lui-même n'a pas changé — seul le framework qui le sert a changé. Chaque route ci-dessus est la même route qu'à l'époque où une seule application Next.js s'en occupait ; c'est juste NestJS qui répond maintenant. Les marqueurs CONSTRUIT sont l'état réel à la date de cette révision — tout le reste de la liste est encore le plan, pas le code.

Assistance automatique — reportée

Planifiée, dimensionnée, et pas construite dès le premier jour.

LA DÉCISION

Il n'y a **aucun machine learning dans ce système au lancement** — pas de modèle, pas d'inférence, rien dans le chemin de requête qui puisse être lent, faux, ou en panne. Il y a un service Python dès la phase 1, délibérément minimal : un conteneur FastAPI avec health-check, aucun modèle chargé et aucune empreinte mémoire significative, existant uniquement pour que la forme du déploiement — un conteneur, joignable, avec health-check, présent dans le fichier compose — soit prouvée tôt plutôt que d'être une surprise de phase 4. La distinction qui compte reste la même : le service existe, mais rien dedans ne prend encore de décision. La classification de dialecte, les embeddings et la traduction automatique restent en **phase 4** — et quand ils arriveront, ils tourneront hors machine, pas sur le serveur d'application.

11.1 Pourquoi c'est reporté

- **C'était le plus gros facteur du coût d'infrastructure.** Un modèle de classe MARBERT pèse 1,5 à 2 Go. Même en ne le chargeant que la nuit, le démarrer pendant que Postgres digère la file est une façon réaliste de se faire tuer par OOM. L'abandonner ne gère pas ce risque, il le supprime (§2).
- **Ça ne fait pas la partie intéressante.** Il n'existe pas de modèle préentraîné au niveau des sous-régions tunisiennes. Un classificateur peut dire qu'une phrase est tunisienne. Il ne peut pas dire qu'elle vient du Kef et pas de Sfax. Ce jugement — la vraie contribution ici — reste humain de toute façon.
- **Impossible de l'entraîner avant que le corpus n'existe.** La classification régionale a besoin de ce jeu de données d'abord. Construire le consommateur avant l'entrée, c'est à l'envers.

11.2 Ce qui fait le travail à la place

Des règles : détection de script, bornes de longueur, un pré-filtre par expression régulière sur des marqueurs, suppression des quasi-doublons (§12.3). Plus faible, et le §12.3 dit exactement ce que ça coûte. Je préfère ça à avoir besoin d'un vrai modèle et d'un GPU dès le premier jour — le service à côté existe déjà (§10.0), il ne fait juste encore rien. La translittération n'est pas affectée — ça a toujours été un moteur de règles (§13).

11.3 À quoi la phase 4 pourrait ressembler

Pas planifié en détail — c'est la phase 4, et je la concevrai correctement quand j'y arriverai. Juste pour que ce ne soit pas complètement vide, en gros ce vers quoi j'irais :

- Un pré-filtre de dialecte — quelque chose de classe MARBERT, tournant hors ligne sur l'arriéré pour réduire la charge des relecteurs, pas pour prendre des décisions.
- Un classificateur régional, entraîné sur ce corpus une fois qu'il sera assez gros — la vraie contribution de recherche, et la raison pour laquelle le corpus doit venir en premier.
- Des embeddings hébergés + pgvector pour l'appariement flou. La recherche exacte et par trigrammes est déjà livrée en phase 2 sans aucun modèle (§14.2), donc ça ne couvre que le cas flou.

Quoi que ça devienne, ça tournera hors machine, facturé à la seconde, pas sur le serveur d'application — de ça je suis sûr, puisque c'est ce qui garde la machine petite (§2). Et chaque fois qu'une sortie de machine arrivera, elle entrera comme une suggestion, `is_machine: true`, jamais directement dans un champ canonique (§7.9, §12.4). Cette règle n'attend pas que les détails ci-dessus soient tranchés.

Ingestion et scraping

L'automatisation existe pour protéger la seule chose que je ne peux pas acheter davantage : l'attention des relecteurs.

YOUTUBE N'EST PAS UNE SOURCE

Les Developer Policies de l'API YouTube exigent que le texte des commentaires, les titres de vidéo et les noms des créateurs soient rafraîchis ou supprimés dans les 30 jours suivant leur stockage — une horloge glissante, permanente, tant que quoi que ce soit est conservé. C'est incompatible avec un corpus construit pour être permanent, et ça contredit directement l'invariant selon lequel `raw_body` est immuable (§7.2, §12.6). Les commentaires YouTube sont exclus comme source. Les forums sont la source de la phase 1 à la place.

12.1 Types de sources

SOURCE	RENDEMENT	DIFFICULTÉ	DROITS
Forums	volume élevé, informel, scripts mélangés	moyenne — parsing HTML	Aucune clause de rétention imposée par une plateforme. La source de la phase 1. Les CGU de chaque site lues avant activation (§12.2).
Contribution directe	faible volume, meilleure qualité	aucune	Propre — consentement explicite. Le seul palier entièrement sûr commercialement.
Sollicitation ciblée	visant des lacunes connues	aucune	Propre.
Livres / imprimé	faible volume, haute qualité, historique	élevée — OCR, RTL, mise en page	Droit d'auteur sauf domaine public. La voie principale vers <i>era</i> : <i>historical</i> .
Sous-titres	moyen, beaucoup de dialogue	moyenne	Généralement sous droit d'auteur. Vérification de licence par source.

LE SCRAPING AGGRAVE EXACTEMENT LE BIAIS QUE CE PROJET EXISTE POUR CORRIGER

Ce qui est en ligne est disproportionnellement nordiste et urbain. Ne scraper que ça, et on obtient un corpus biaisé vers Tunis — garanti. Le contrepoids est la **sollicitation ciblée** : le système trouve des entrées de lexique sans variante du sud, ou des phrases sans équivalent du nord-ouest, et demande directement aux gens de ces régions. Une lacune devient une question à laquelle quelqu'un peut répondre en dix secondes au lieu d'un vague appel à l'aide.

12.2 Ce qui est scrapé et ce qui ne l'est pas

SOURCE	STATUT	POURQUOI
Forums tunisiens	En cours d'utilisation	Récupération HTML sous les CGU propres à chaque site, pas un contrat d'API de plateforme avec une clause de rétention attachée. Les CGU de chaque site sont lues avant l'activation.
Blogs, sites personnels	En cours d'utilisation	Mêmes conditions que les forums. Le droit d'auteur ordinaire s'applique à ce qui est stocké (§12.6).
Commentaires de sites d'actualités	Au cas par cas	CGU lues par site, même processus que pour les forums.
Wikipédia en arabe tunisien (aeb.wikipedia.org), Wiktionnaire	En cours d'utilisation	CC BY-SA, dumps explicites publiés pour la réutilisation, aucune exigence de rafraîchissement, aucun risque de limitation de débit. Petit mais déjà segmenté et déjà écrit en derja en script arabe — bon pour valider le pipeline avant de se battre avec une API de plateforme.
Common Crawl, filtré aux domaines .tn	En cours d'utilisation	Sous licence permissive exactement pour ça, énorme, aucune relation de plateforme à entretenir.
Tatoeba	En cours d'utilisation	Paires de phrases CC-BY, une partie en arabe tunisien, déjà parallèles — directement utiles pour le traducteur.
YouTube (API)	Exclu	La règle des 30 jours de rafraîchissement/suppression (§12.1) est incompatible avec un corpus permanent et contredit directement l'immutabilité de raw_body. Tranché, pas un « pour l'instant » — point final.
Facebook / Instagram	Exclu	Les CGU interdisent la collecte automatisée ; Meta poursuit les scrapers en justice.
Reddit	Exclu	API payante avec ses propres conditions de rétention et d'usage commercial depuis 2023.
Twitter/X (API)	Exclu	Payant, conditions de stockage et de redistribution restrictives.
TikTok	Exclu	API développeur propre avec conditions commerciales ; pas rentable à cette échelle.

Les trois ajouts depuis la première version de ce document — Wikipédia, Common Crawl, Tatoeba — sont ce qui a vraiment fini dans la liste des sources de la phase 1 une fois que j'ai cherché des sources ouvertes, basées sur des dumps, plutôt que seulement des sources façonnées par des plateformes. L'argument « la licence voyage avec les données » ci-dessous est la vraie raison pour laquelle elles battent les API de plateformes du côté exclu, pas juste une question de permission sur le papier.

POLITIQUE DE DROITS POUR LE TEXTE SCRAPÉ

Chaque source est vérifiée contre ses conditions d'utilisation, pas seulement son robots.txt — ce sont deux documents séparés et la permission de l'un n'implique pas celle de l'autre. Le texte scrapé et stocké est traité comme réservé à un usage recherche et exclu du palier commercialement propre par défaut (§4.4), quelle que soit la source. La protection des données suit la posture RGPD du §4, puisque l'hébergement se trouve dans l'UE.

Pourquoi les dumps ouverts battent les API de plateformes, dit une fois pour ne pas l'oublier. La licence doit voyager avec les données. Un dump CC-BY-SA téléchargé une fois m'appartient, à garder et republier indéfiniment. Tout ce qui est accédé via une API de plateforme porte la laisse de cette plateforme pour toujours — obligations de rafraîchissement, autorisation révocable, conditions qui peuvent changer sous mes pieds avec 30 jours de préavis. Un corpus censé me survivre ne peut être construit de façon fiable qu'à partir du premier type — c'est la vraie raison pour laquelle Wikipédia, Common Crawl et Tatoeba sont dedans et YouTube, Reddit et Twitter/X sont dehors, pas juste que les API coûtent de l'argent.

La logique de filtrage qui décide ce qu'un scraper garde ou jette n'est pas encore tranchée non plus — ce qui suit est la forme générale, pas un jeu de règles final. Il faut s'attendre à ce que les seuils et les marqueurs réels bougent une fois que de vraies données commenceront à arriver.

12.3 Étapes du pipeline

```

récupération brute
-> stockage tel quel dans documents.raw_body (jamais retouché ensuite)
-> nettoyage (retrait des urls, mentions, emoji, normalisation des espaces)
-> filtre de longueur (3–500 tokens)
-> détection de script
-> pré-filtre par expression régulière (marqueurs lexicaux spécifiquement tunisiens)
-> suppression des quasi-doublons (MinHash LSH, seuil 0,85)
-> découpage en éléments de corpus (paragraphe -> phrase)
-> calcul du match_key (§13.5)
-> insertion des éléments de corpus, attribution du dataset_split
-> génération des tâches de relecture

[phase 4] classification de dialecte – par lots hors ligne, s'intercale avant le découpage en élémén

```

FATIGUE DE LA FILE DE RELECTURE — CE QUE COÛTE L'ABANDON DU CLASSIFICATEUR

Les règles seules laissent passer plus de bruit qu'un modèle ne le ferait. Meilleure estimation : abandonner un filtre de dialecte par apprentissage automatique fait monter la fuite de bruit — l'arabe standard et d'autres dialectes maghrébins qui ne sont pas vraiment du derja tunisien — d'environ 20–30 % à 40–50 % de ce qui atteint la relecture. C'est une charge de relecture manuelle réelle et nettement plus lourde à travers les phases 1 à 3, pas une erreur d'arrondi.

La solution, c'est le débit, pas la finesse : petits lots d'ingestion, la liste de marqueurs conservée comme donnée versionnée pour qu'elle s'améliore grâce aux vrais rejets, et le taux de rejet d'une source traité comme une raison de la désactiver. La question de savoir si ça vaut le coup de faire venir un filtre de dialecte plus tôt que la phase 4 pour réduire cette fuite plus vite reste ouverte — notée ici plutôt que tranchée.

LE SCRAPING NE SERT PAS L'ARGUMENT DU SIGNAL RÉGIONAL — UNE VRAIE LIMITE, PAS UNE NOTE DE BAS DE PAGE

Une phrase scrapée n'a aucune métadonnée de région. Aucune. Alors comment un relecteur la tague-t-il northwest ? En reconnaissant des marqueurs du nord-ouest dans le texte — ce qui veut dire que l'étiquette régionale sur le contenu scrapé encode ce que les relecteurs croient déjà être régional, pas une preuve indépendante que ça l'est vraiment. C'est circulaire. Nourrir ça à un classificateur plus tard (§11.3), et j'aurai entraîné un modèle à reproduire les a priori de mes relecteurs et appelé ça une découverte.

Le matériau contribué est d'une nature différente : quelqu'un dit « je suis du Kef, chez nous on le dit comme ça. » Preuve faible, mais *indépendante* des traits linguistiques que j'essaie d'apprendre — un témoignage, pas une inférence.

Conséquence, comme politique : les éléments de corpus scrapés partent par défaut sur `region: unknown` plutôt que d'être tagués par intuition linguistique. Un relecteur ne peut taguer la région d'un élément scrapé que lorsqu'un contributeur de cette région a indépendamment attesté la même variante. Le scraping achète du volume et une couverture générale du derja. Il ne peut pas, à lui seul, acheter des revendications régionales — la sollicitation ciblée est le pipeline porteur de ce sur quoi ce projet se différencie vraiment.

12.4 La déduplication s'applique aussi aux contributions

MinHash se trouve dans le pipeline du scraper, ce qui laisse un trou évident : quelqu'un soumet une phrase déjà dans le corpus, et elle passe comme du nouveau travail qu'un relecteur devra rejeter.

Donc la même vérification de quasi-doublon tourne sur le chemin de contribution, contre `match_key` — et la réponse est la partie intéressante. Pas « rejeté, doublon », qui sonne comme une gifle. Plutôt : *on a déjà celui-là — mais personne n'a enregistré comment le sud le dit. Tu sais, toi ?* Ça transforme un rejet en invitation à contribuer, le même tour que le §14.4 joue avec les échecs du traducteur.

12.5 Rester dans les règles, pas juste éviter un bannissement

Deux problèmes différents vivent ici et ont besoin de deux solutions différentes. Se faire bannir par IP est une nuisance opérationnelle. Violer les conditions d'utilisation d'une plateforme, ou une loi de protection des données, est un problème d'une autre nature — la solution n'est pas un proxy, c'est de ne pas scraper cette source du tout (§12.2).

- **Chaque source a ses CGU lues avant sa première exécution**, pas seulement son robots.txt vérifié. Ce sont deux documents différents répondant à des questions différentes, et un site peut autoriser les robots dans l'un tout en interdisant le stockage dans l'autre.
- **Un serveur veut dire une seule IPv4**, donc scraper en volume depuis une seule adresse risque un bannissement même sur une source pleinement autorisée. Prévoir la rotation de proxys pour tout ce qui est parsé en HTML dès le départ, plutôt que la découvrir la semaine après le premier bannissement.
- **Les limites de débit et le backoff sont une config par source**, sur la ligne de la source, modifiable depuis le panneau admin, backoff exponentiel sur les erreurs 429 et 403.
- **Deux exécutions ratées désactivent la source** et lèvent une alerte (§17.1), au lieu de continuer à marteler un hôte qui a déjà dit non.
- **Scraper lentement et constamment** plutôt que par rafales. Il n'y a pas d'échéance — ce projet est censé s'accumuler pendant des années, et être poli est à la fois le choix conforme et le choix le moins cher opérationnellement.

12.6 Règles

- Le matériau brut est immuable. Chaque étape lit une représentation et en écrit une nouvelle — ce qui explique exactement pourquoi une source avec une clause de rafraîchissement/suppression obligatoire (§12.1) ne peut pas être une source ici.
- La sortie d'une machine — chaque fois qu'elle existe — entre comme une suggestion, jamais comme un fait.
- Chaque source est enregistrée avec sa licence par défaut et son statut de CGU issu du §12.2, pour que les droits s'attachent à l'ingestion plutôt que d'être devinés plus tard.
- Les scrapers respectent le robots.txt, les limites de débit, et les conditions d'utilisation du site — les deux premiers ne remplacent pas le troisième.
- Les identifiants d'auteur sont retirés à l'ingestion. Aucune donnée personnelle issue de sources sociales, jamais (§4.4).

Standardisation, translittération, appariement

Trois métiers différents sur le même texte. Les garder séparés est tout le truc.

13.1 Ne pas inventer une orthographe

CODA-TUN (LREC 2014) et NOTA définissent déjà une orthographe conventionnelle pour l'arabe tunisien, et CODA* maintient une base de données orthographique vérifiée à travers les dialectes. Les utiliser bat inventer autre chose : relu par les pairs, gratuit, et ça fait fonctionner ce jeu de données avec toute la chaîne d'outils CAMEL et MADAR. Inventer un standard rival coûte des mois et rend les données moins utiles au bout du compte.

13.2 Les règles sont des données versionnées, pas du code

```
standardisation_rules
  id, rule_version, pattern, replacement, script, description, active
```

Une règle orthographique changera au bout de six mois. Si les formes canoniques sont écrasées sur place sans version enregistrée, chaque instantané déjà publié devient irreproductible et incohérent en interne. Les règles comme lignes versionnées rendent la régénération déterministe — et rejouable à une version antérieure, ce qui est ce qui rend un mauvais changement de règle récupérable (§17.2).

13.3 Correspondance arabizi

L'arabizi vers l'arabe n'est pas une correspondance un-à-un. Des chiffres remplacent des lettres sans équivalent latin, et plusieurs correspondances dépendent du contexte.

ARABIZI	ARABE	ARABIZI	ARABE	ARABIZI	ARABE
2	ء	3	ع	5	خ
6	ط	7	ح	8	غ
9	ق				

AMBIGU	PEUT ÊTRE	REMARQUE
« e »	ه / ي	ou muet
« a »	ة / ا	ou une fatha
« ch »	ش	influence française — pas « sh »
« ou »	و	influence française
« th »	ث / ذ	selon le contexte

Ce tableau mérite d'être publié à part — utile aux gens qui se moquent du reste du corpus, et une façon peu coûteuse d'avoir l'air de savoir ce que je fais dès le début.

13.4 Le flux de standardisation

1. Le moteur de règles produit une forme canonique suggérée et marque les intervalles dont il n'est pas sûr.
2. Le relecteur voit la suggestion avec ces intervalles surlignés et les corrige en ligne.

3. La forme acceptée est stockée avec son `rule_version` et avec `canonical_map` — l'alignement d'offsets entre l'original et la forme canonique, que le moteur a déjà calculé en appliquant ses règles. Cette carte garde les intervalles de tokens (§7.2) utilisables contre les deux versions du texte.
4. Les corrections qui reviennent souvent sont des règles candidates. L'interface du relecteur fait remonter les corrections manuelles les plus fréquentes.

13.5 La clé d'appariement — une troisième représentation

POSTGRES N'A PAS DE CONFIGURATION DE RECHERCHE TEXTUELLE EN ARABE

Ça compte et c'est facile à rater. Postgres livre des configurations de recherche textuelle pour l'anglais, le français, l'espagnol, le turc et une douzaine d'autres. Il n'y en a pas pour l'arabe. `to_tsvector('arabic', ...)` n'existe pas. Tout plan qui dit « on utilisera la recherche plein texte » pour le traducteur signe un chèque que la base de données ne peut pas honorer.

Trois options, et ce document en choisit une plutôt que de laisser la question ouverte :

OPTION	VERDICT
config simple + normalisation maison	Fonctionne. Pas de racinisation, donc <code>كتب</code> ne correspondra pas à <code>يكتب</code> . Pour la recherche dans le lexique, c'est surtout correct — je fais correspondre des formes attestées, pas du texte libre fléchi.
pg_trgm	Trigrammes de caractères, agnostique à la langue, vraiment bon pour la tolérance aux fautes de frappe et les variations orthographiques — ce qui est l'essentiel de ce qu'est réellement la recherche en derja. Le cheval de bataille.
Racinisateur arabe Snowball	Existe, calibré pour l'arabe standard. Une racinisation d'arabe standard appliquée au derja sera faussée assez souvent pour faire mal. Non.

Décision : pg_trgm plus la config simple, pas de racinisateur. Le derja n'a pas de morphologie standard sur laquelle racinifier, et je fais correspondre contre des formes canoniques que je contrôle.

Normaliser avant toute comparaison

Sans ça, le traducteur rate constamment, parce que l'arabe a plusieurs façons d'écrire la même chose et les gens les utilisent toutes. Avant d'indexer ou de comparer :

```
variantes de alif fusionnées | <- ا آ إ | ا
taa marbuta -> haa         | <- ة
alif maqsura -> yaa        | <- ي
diacritiques retirées      | fatha, damma, kasra, shadda, sukun
tatweel retiré             | -
chiffres arabes-indiens    | ٩-٠ -> 0-9 (l'arabizi mélange les deux)
```

CE N'EST PAS LE MÊME MÉTIER QUE LA STANDARDISATION CODA

CODA produit la forme canonique qui est *stockée et affichée*. Ceci produit une clé d'appariement que personne ne voit jamais. Les confondre est un vrai piège : on veut que la recherche soit agressive et tolérante, et on ne veut absolument pas que cette agressivité fuite dans les formes canoniques publiées. Deux couches, deux buts, deux colonnes.

```
corpus_items.match_key      text  -- normalisé, indexé, jamais affiché
lexicon_variants.match_key  text

lexicon_forms.match_key     text
index gin (match_key gin_trgm_ops)
```

Le traducteur

La raison pour laquelle quiconque ouvre le site. Livré en phase 2, n'a besoin d'aucun modèle.

14.1 Ce que c'est

- **Derja** → **arabe standard / français / anglais**. N'importe quel script, n'importe quelle région.
- **Arabe standard / français / anglais** → **derja, avec un sélecteur de région**. Personne d'autre n'offre cette direction. « Dis ça en derja du sud » est quelque chose qu'aucun outil existant ne sait faire, et c'est la démo qui pousse les gens à partager le lien.

Le résultat n'est pas une seule chaîne. Ce sont les formes attestées avec leurs régions, leurs scripts, un lien vers l'élément de corpus d'où elles viennent, et — là où quelqu'un l'a enregistré — d'où vient le mot (§7.6). Ce dernier détail ne coûte rien et fait beaucoup de travail : voir qu'un mot qu'on utilise tous les jours est arrivé via le turc ottoman, c'est ce qui fait que la page ressemble à un ouvrage de référence plutôt qu'à un simple dictionnaire.

14.2 Ça n'a besoin d'aucun modèle

- **Mot et phrase** — recherche via `lexicon_entries` → `lexicon_variants` → `lexicon_forms`. Correspondance exacte sur `match_key`, puis correspondance normalisée via les règles de standardisation, puis similarité `pg_trgm` pour les fautes de frappe et les variations orthographiques (§13.5).
- **Phrase** — similarité par trigrammes sur `corpus_items.match_key`, retournant les éléments de corpus attestés avec leurs traductions humaines. Pas de la recherche plein texte, pour la raison expliquée au §13.5.
- **Filtrage par région** — une jointure `lexicon_variant_regions`. Déjà dans le schéma.

Pas d'embeddings, pas de clé d'API, pas de coût, rien dans le chemin de requête qui puisse tomber en panne.

14.3 Pourquoi ça s'améliore tout seul

LA PROPRIÉTÉ QUI FAIT QUE C'EST LA BONNE FORME

Chaque annotation acceptée améliore le traducteur au moment même où elle est écrite. Pas de réentraînement, pas d'instantané à promouvoir, pas de risque de régression. Un modèle entraîné s'améliore par sauts versionnés et peut empirer ; un traducteur par recherche ne peut que s'améliorer, et il s'améliore le jour même où quelqu'un contribue. Ça vaut plus ici que la fluidité — ça fait de l'outil une raison vivante de contribuer au lieu d'une chose qui consomme des contributions.

La page le dit, avec un compteur en direct : *ce traducteur connaît N mots et M phrases à travers quatre régions, en hausse depuis X le mois dernier*. Le premier jour, ce compteur affiche quelque chose d'embarrassamment petit, et je le laisse visible quand même, parce que « 12 mots, en croissance » est un argumentaire plus honnête qu'une page d'accueil qui fait semblant d'être terminée.

14.4 Échouer honnêtement, et la boucle qui en sort

Quand il n'y a pas d'attestation, le traducteur ne devine pas. Il montre quels mots il a trouvés et lesquels non, les voisins attestés les plus proches marqués comme approximatifs, ce qui manque par région — « *attesté au nord et au sahel. Aucune attestation du sud.* » — et une invitation à contribuer en un clic pour exactement cette lacune.

CHAQUE RECHERCHE INFRUCTUEUSE EST UNE INVITATION À CONTRIBUER

Les lacunes du traducteur *sont* la page `/gaps`, générée par la demande réelle plutôt que devinée. Quelqu'un du sud qui cherche un mot et voit « personne n'a enregistré comment le sud dit ça » est la meilleure invitation à contribuer de tout le système, et ça ne coûte rien à produire.

14.5 Montée en puissance

PHASE	CAPACITÉ	BESOINS
2	Recherche de mot et de phrase, exacte + normalisée + trigramme, filtrée par région	lexique + pg_trgm
2	Récupération de phrases par similarité de trigrammes	match_key + index GIN
3	Direction inverse avec sélecteur de région	couverture du lexique dans les quatre régions
4	Appariement sémantique flou	embeddings hébergés + pgvector
5	Génération ancrée pour les entrées non attestées	corpus + API, étiquetée comme générée

14.6 Ce que ce n'est pas

La page dit ce qu'elle est : une recherche dans un corpus, pas de la traduction automatique. Elle retourne ce que les Tunisiens ont vraiment écrit, ou elle ne retourne rien. C'est un produit plus faible qu'un modèle fluide, et plus honnête, et c'est cette honnêteté qui permet à un chercheur de le citer.

La même prudence couvre tout ce qui est généré. Un modèle entraîné seulement sur ce corpus sera mauvais pendant longtemps, et en mettre un devant les gens trop tôt brûle la crédibilité auprès exactement du public dont j'ai besoin. La génération, c'est la phase 5, étiquetée, jamais mélangée aux résultats attestés. Quand des modèles arriveront, ils s'entraîneront contre des instantanés figés et ne seront promus que sur une amélioration mesurée contre un ensemble de test qui n'entre jamais dans l'entraînement — pas de réentraînement continu, parce que ça ne donne aucune reproductibilité, aucun moyen de repérer une régression, et absorbe discrètement les mauvaises annotations.

CE QUE ÇA COÛTE : LE LEXIQUE ATTERRIT EN PHASE 2

lexicon_entries, lexicon_variants, lexicon_variant_regions, lexicon_forms et le lien de lemmes sont ce sur quoi tourne le traducteur, donc ils appartiennent à la phase 2 plutôt qu'au reste de la couche de service. Le lien de lemmes est un travail d'annotation central ici, pas une gentillesse tardive.

Frontend et système de design

C'est un projet de langue. L'interface a intérêt à être bonne en matière de langue.

15.1 Stack

ASPECT	CHOIX	POURQUOI
Framework	Next.js App Router	Les pages publiques ont besoin de SSR pour la recherche et la citation ; les tableaux de bord sont lourds côté client. Un seul framework couvre les deux.
Style	Tailwind + variables CSS pour les tokens	RTL via des propriétés logiques, pas des bricolages de direction.
Récupération de données	Composants serveur en public, TanStack Query pour les tableaux de bord	Les tableaux de bord ont besoin de mises à jour optimistes et de refetch en arrière-plan. Pas les pages publiques.
Formulaires	react-hook-form + zod (schémas partagés)	Même validation des deux côtés, une seule définition.
Tableaux	TanStack Table	Les files de relecture ont besoin de virtualisation, de tri, de contrôle des colonnes.

15.2 Bilingue et bidirectionnel dès le départ

Le contenu arabe se trouve constamment à l'intérieur d'une interface anglaise ou française. La direction mixte est le cas normal ici, et l'ajouter après coup coûte bien plus cher que de le faire dès le début.

- Propriétés CSS logiques partout — `margin-inline-start`, jamais `margin-left`.
- Chaque nœud de texte en derja reçoit `dir="rtl"` et `lang="ar-TN"`, quoi que fasse l'interface autour.
- Marques LRM/RLM autour du contenu inline à scripts mélangés, sinon l'algorithme bidi le réordonne en charabia.
- Une police avec une vraie couverture arabe. Les polices système par défaut rendent mal le texte tunisien.
- L'arabizi est en script latin mais c'est une langue tunisienne : `lang="aeb-Latn"`, LTR. Ni l'un ni l'autre des deux autres cas.

Trois locales d'interface : anglais, français, arabe. Le français compte plus qu'il n'y paraît — beaucoup d'utilisateurs tunisiens lisent le texte d'interface plus confortablement en français qu'en arabe standard. La locale est indépendante de la direction du contenu.

15.3 Composants d'annotation

Les tableaux de bord partagent un petit ensemble de composants. Bien les construire une fois, c'est l'essentiel du travail frontend.

COMPOSANT	CE QU'IL FAIT
<CorpusItemText>	Bon dir/lang, surligne les intervalles tagués/liés, sélection pour en créer de nouveaux
<SpanSelector>	Sélection souris et tactile retournant des offsets de caractères, accroche aux frontières de tokens
<TagChip>	Genre + valeur, coloré par catégorie, supprimable, annotateur au survol
<RegionPicker>	Quatre régions, sélection multiple (§6.4), plus pan-tunisien et incertain. Touches 1–5.
<TaskCard>	Enveloppe n'importe quel type de tâche, gère réclamer/compléter/passé de la même façon pour toutes
<ScriptToggle>	Bascule un élément de corpus entre vues arabe et arabizi
<DiffView>	Original contre standardisé, surlignage au niveau caractère

Construit jusqu'ici : aucun de ces sept composants n'existe encore. Les 31 routes frontend du §16 sont de purs squelettes — un titre et une ligne de description chacune, juste assez pour que `next build` compile et que chaque route retourne 200, rien de branché à de vraies données. Tout cet ensemble de composants reste à faire.

15.4 Clavier d'abord pour les relecteurs

Quelqu'un qui traite des centaines d'entrées d'une traite est limité par les déplacements de souris. Le fonctionnement complet au clavier est une fonctionnalité de débit, pas une case d'accessibilité à cocher, et c'est conçu comme tel dès le départ.

1-5	régler la région	a	approuver
j / k	suivant / précédent	r	rejeter
entrée	soumettre + suivant	s	passer
t	focus traduction	e	éditer la forme standardisée
?	aide raccourcis	f	marquer pour adjudication

Chaque page, spécifiée

Avec la phase associée, pour que rien ne soit construit trop tôt.

Voici les écrans de base tels que j'en comprends le besoin en ce moment. Je m'attends à trouver des pages à ajouter, séparer, fusionner ou abandonner une fois que je serai vraiment en train de construire et de les utiliser — cette liste est un point de départ, pas une spec figée.

16.1 Pages publiques

PAGE	ROUTE	CE QU'IL Y A DESSUS	PHASE
Accueil	/	Ce que c'est, compteurs en direct par région, un exemple de comparaison, appel à contribuer	2
Traduire	/translate	Traducteur en direct, les deux directions, conscient de la région, honnête sur ce qu'il n'a pas (§14)	2
Explorer	/explore	Recherche et filtres : région, portée, époque, cadre, registre, unité, script, a-une-traduction	2
Comparer	/compare	Une entrée de lexique, quatre colonnes régionales. La page que les gens partagent.	2
Permalien d'entrée	/s/:id	Texte, tags, traductions, attestations, source, historique des révisions, bouton citer, lien de signalement (§16.4)	2
Lexique	/lexicon	Liste des entrées de lexique, variantes régionales, comptes d'attestations	2
Entrée de lexique	/lexicon/:id	Une entrée de lexique, chaque variante et forme, exemples d'attestations, origine avec note de dérivation (§7.6)	2
Origines	/origins	Parcourir par strate — chaque mot d'origine amazighe, chaque emprunt italien, la note sur le suffixe -ji — et ajouter une origine soi-même si on en connaît une non enregistrée (§7.6)	2
Recherche	/research	Bibliographie des travaux sur le derja tunisien. Soumise par la communauté, amorcée avant le lancement (§7.10)	2
Téléchargement	/data	Versions publiées, formats, checksums, licence par palier, journal des changements	2
Méthodologie	/methodology	Sourcing, adoption de CODA, processus d'annotation, taux d'accord, biais connus	2
Couverture	/coverage	Équilibre régional, distribution par unité, complétude des traductions, lacunes	2
Guide de style	/style	Conventions orthographiques, le tableau de correspondance arabizi, exemples travaillés	2
Citation	/cite	BibTeX, DOI Zenodo, chaînes de citation par version	2
Licence	/license	Droits par source, quels paliers sont utilisables commercialement	2
Lacunes	/gaps	Les plus demandés, guidés par les échecs du traducteur et les déficits de couverture	3
Classement	/contributors	Par nombre approuvé, et par région	3
Soutien	/support	Dons et parrainage seulement — rien à vendre, rien de bloqué derrière. À quoi servirait l'argent.	3
Docs API	/docs/api	Points d'accès, authentification, limites de débit, exemples	4

16.2 Pages contributeur — une question, trois secondes

Chacune de ces pages est construite contre le §9.6 : aucune n'expose les niveaux de confiance, les types de tâches, ou quoi que ce soit de la mécanique derrière.

PAGE	ROUTE	CE QU'IL Y A DESSUS	PHASE
Contribuer	<code>/contribute</code>	Une case : écrire un mot, une phrase, ou un paragraphe. Région comme quatre boutons cliquables, consentement comme une case à cocher, pas de compte. Traduction, script et détail de région sont des extras optionnels sous le pli — les sauter et la file de tâches de quelqu'un d'autre rattrape exactement ce qui manque (§8).	3
Confirmer	<code>/confirm</code>	Un élément, une question, trois boutons, rien d'autre sur la page. La surface la moins coûteuse en effort qui existe — c'est ce qui transforme un visiteur en contributeur.	3
Boîte de tâches	<code>/tasks</code>	Les tâches ouvertes que cette personne peut faire. Ne montre jamais un état vide ou verrouillé — s'il n'y a rien, ça redirige vers <code>/contribute</code> .	3
Tâche de traduction	<code>/tasks/translate</code>	Élément de corpus plus brouillon, corriger et soumettre	3
Translittérer	<code>/tasks/transliterate</code>	Élément de corpus plus suggestion générée par règle, intervalles incertains surlignés	3
Proposer une correction	<code>/s/:id/correct</code>	Suggérer une correction sans accès en écriture	3
Mes contributions	<code>/me</code>	Ce qu'ils ont contribué et si c'est accepté. Pas un niveau, pas un palier, pas un parcours.	3

16.3 Pages relecteur — connecté

PAGE	ROUTE	CE QU'IL Y A DESSUS	PHASE
File de relecture	<code>/review</code>	Triage : approuver, rejeter, tag de région, tag de qualité. Piloté au clavier.	1
Lien de lemmes	<code>/review/lemma</code>	Rattacher des tokens à des variantes de lexique. Travail d'annotation central — le traducteur tourne dessus.	2
Adjudication	<code>/review/adjudicate</code>	Tags en conflit côte à côte avec qui les a faits ; choisir ou passer outre. Routée par région (§8.5).	3
Corrections	<code>/review/corrections</code>	Corrections communautaires, propositions d'origine, soumissions de publications, signalements	3
Standardisation	<code>/review/standardise</code>	Suggestion contre original, intervalles incertains, accepter ou éditer	4

16.4 Deux choses que les pages publiques doivent trancher

Signalement

Le §4.4 promet que les demandes de retrait sont honorées rapidement, ce qui est difficile sans moyen d'en faire une. Chaque permalien porte un lien de signalement qui écrit dans `flags` (§7.8), alimentant la file des corrections. Je vais recevoir des insultes, des propos offensants, le numéro de téléphone de quelqu'un, et du contenu qu'une personne veut voir disparaître — ce n'est pas du pessimisme, c'est ce qui arrive à tout site qui accepte du texte public. Un bouton coûte moins cher qu'une boîte de réception.

Registre vulgaire

`register` : vulgare existe parce que le derja en est plein et l'exclure falsifierait le corpus. Mais `/explore` et `/translate` sont publics et non authentifiés, donc la politique doit être énoncée plutôt que laissée à qui écrit la requête :

- **Filtré par défaut, avec un interrupteur opt-in visible.** L'interrupteur est honnête sur ce qu'il fait — il ne dit pas « mode sûr », il dit « inclure le registre vulgaire ». La préférence vit dans un cookie, même idée que le cookie de session au §9.3 — pas besoin de compte pour garder le choix entre deux visites.
- **L'API porte le même interrupteur, pas une politique séparée.** `/v1/search` et `/v1/lookup` prennent tous les deux un paramètre `include_vulgar` (§18.1) qui fait exactement ce que fait l'interrupteur de l'interface. Sans lui, les deux sont filtrés par défaut — l'API n'a pas une règle plus discrète ou plus laxiste que la page.
- **Toujours présent dans les exports,** toujours étiqueté. Un corpus qui laisse discrètement tomber la moitié de la façon dont les gens parlent vraiment n'est pas un corpus de la façon dont les gens parlent.
- **Jamais filtré dans la file de relecture.** Les relecteurs voient tout ; c'est le travail.

Le tableau de bord admin

Cette page décide si une seule personne peut vraiment faire tourner ce projet.

Le public entier de cette page, c'est moi, à des heures indues, probablement un peu paniqué. Tout ce qui y figure mérite sa place selon ce critère.

17.1 Aperçu — le seul écran qui répond à « est-ce que ça va »

WIDGET	MONTRE	ALERTE QUAND
Débit du pipeline	ingéré / relu / publié, 24h et 7j	taux de relecture sous le taux d'ingestion pendant 3 jours
Santé de la file de tâches	ouvertes par type, la plus ancienne ouverte, temps moyen pour compléter	quoi que ce soit de plus vieux que 14 jours
Équilibre régional	part en direct par région contre l'objectif	une région sous 15 % du total
Accord des annotateurs	taux d'accord glissant, retard d'adjudication	accord sous 0,7
Adjudication hors région	part décidée par un relecteur d'ailleurs (§8.5)	plus de 20 % pour une région — un problème de recrutement, pas un seuil à relâcher
Couverture du traducteur	taux de succès des recherches, taux d'échec par région	le taux d'échec d'une région dépasse 60 % — la lacune où le corpus échoue, mesurée par la demande réelle
Activité des contributeurs	contributeurs actifs sur 7j, nouveaux contre récurrents	aucun nouveau contributeur en 7 jours
Statut des scrapers	dernière exécution par source, rendement, taux d'erreur	une source qui échoue deux fois de suite
Santé des services	statut des conteneurs nlp et media, disponibilité (le backend lui-même tourne sous pm2, §10.0)	l'un des deux en panne plus de 5 minutes
Système	taille de la base, disque, mémoire, profondeur de la file, taux d'erreur, dernière sauvegarde	disque au-dessus de 80 %, profondeur de file au-dessus de 1000, sauvegarde de plus de 48h

Chaque « taux d'erreur » sur cet écran est adossé à Sentry (§10.0) — DSN configuré, échantillonnage de traces par défaut — pas un chiffre inventé pour le tableau de bord.

Cet écran porte chaque chiffre que le §9.6 tient à l'écart des contributeurs. C'est le but — la complexité est réelle et entièrement instrumentée, elle fait juste face à moi au lieu d'eux.

17.2 Sections

Gestion des données

- Parcourir chaque table avec des filtres — une vue en lecture seule sûre
- Opérations en masse : relancer la standardisation sur une sélection, réattribuer une région, rejeter en masse un mauvais lot de scraping
- Fusionner des entrées de lexique en double ; scinder une entrée de lexique qui mélangeait deux sens
- Chaque opération en masse écrit une ligne `audit_log` avec un diff réversible

- Journal des échecs du traducteur — les termes non attestés les plus recherchés, classés, exportables comme invitations à contribuer

Files de relecture

- Les corrections communautaires, les propositions d'origine, les publications soumises et les signalements empruntent tous la même file `corrections` plutôt que d'avoir chacun leur propre file — les signalements ont une simple vue en liste dès la phase 2 (§20), pliés proprement dans cette file unifiée une fois que le reste existe en phase 3
- Les commentaires de publication en attente d'approbation, et l'interrupteur qui désactive entièrement les commentaires (§7.10)

Administration des tâches

- Reprioriser les files — pousser les tâches des régions du sud quand l'équilibre glisse
- Libérer de force les réclamations bloquées, réattribuer, générer manuellement des tâches pour une sélection
- Mettre entièrement en pause un type de tâche — suspendre la traduction pendant que la standardisation rattrape son retard
- Routage d'adjudication : revoir les cas hors région, ajuster la fenêtre de 7 jours, voir le taux par région
- Régulation de la file : intervalle de sondage, concurrence, taille des lots, en direct (§10.3)

Sources

- Ajouter, éditer, désactiver ; licence par défaut et statut de consentement par source
- Planification, limite de débit, pool de proxys, historique de rendement
- Exécution manuelle ; dernière erreur avec sa stack trace
- Mettre une source en quarantaine et signaler en masse tout ce qu'elle a produit, si les droits s'avèrent troubles
- Les échecs de source et les health-checks nlp/media (la ligne Santé des services du §17.1) alertent via le même chemin Sentry — un seul mécanisme de notification, pas deux

Utilisateurs et contributeurs

- Promouvoir en relecteur ; ajuster la confiance manuellement avec une raison enregistrée
- Qualité par contributeur : taux d'approbation, taux de désaccord, régions d'où ils soumettent
- Bannir par session ou `ip_hash` ; annuler en masse tout ce qu'un mauvais acteur a soumis
- Inviter des relecteurs par email avec des tokens expirants, en capturant leur région

Règles de standardisation

- CRUD complet, avec un aperçu montrant combien d'éléments de corpus un changement toucherait *avant* de l'appliquer
- Incrémenter `rule_version`, puis régénérer les formes canoniques et les cartes d'offsets comme tâche de fond
- Rapport de corrections manuelles fréquentes : ce que les relecteurs corrigent constamment à la main — les règles candidates

UN MAUVAIS CHANGEMENT DE RÈGLE A BESOIN D'UN CHEMIN DE RETOUR

Régénérer chaque forme canonique à partir de nouvelles règles est exactement l'opération dont on découvre après coup qu'elle était fausse. Donc : **instantané avant de régénérer**, garder les lignes `rule_version` précédentes actives plutôt que de les supprimer, et rendre la régénération rejouable à n'importe quelle version antérieure. Faire marche arrière devient alors rejouer une ancienne version, pas restaurer une sauvegarde à 3h du matin.

Instantanés et publications

- Créer : geler, calculer le checksum, compter, notes de version
- Choisir les paliers d'export — complet, propre commercialement, étymologie, par région
- Pousser vers Hugging Face et le stockage objet via des URLs présignées, jamais à travers le processus NestJS — le même mécanisme dont le stockage média du §7.11 aura besoin plus tard

- Comparer deux instantanés : éléments de corpus ajoutés, changés, retirés ; delta de version de règle
- Déprécier avec une raison, affichée publiquement sur la page de téléchargement

Opérations de confidentialité

- Traiter une révocation de consentement avec confirmation tapée (§4.3)
- Prévisualiser quels champs de quelles tables une révocation va nettoyer, avant de l'exécuter
- Demande d'accès aux données personnelles : exporter tout ce qui est détenu sur une session ou un compte
- Journal des révocations — action, horodatage, décompte, et aucun identifiant dedans

Système

- Statut des tâches, journal des échecs, nouvel essai manuel
- Statut des sauvegardes : dernier dump réussi, taille, destination, chiffrement confirmé, dernier test de restauration — alimenté par `verify-backup-restore.yml` (§2.3), pas un rappel dans mon agenda à vérifier manuellement
- Feature flags — activer une page pour les relecteurs avant qu'elle ne soit publique
- Mode maintenance avec une bannière publique
- Journal d'audit complet, cherchable par acteur, entité, action, date

Gestion de modèles — phase 4 seulement

Enregistrer un modèle contre un instantané avec ses métriques ; comparer un candidat contre la production sur l'ensemble de test réservé ; promouvoir ou revenir en arrière. Vide jusqu'à ce que le §11 arrive — spécifié maintenant pour que la discipline de split figé soit conçue dès le départ plutôt que greffée après coup.

RÈGLES POUR TOUT CE QUI EST DESTRUCTEUR

Rien n'est jamais supprimé pour de vrai. Chaque action destructrice est une suppression douce plus une entrée d'audit. Les opérations en masse au-dessus d'un seuil demandent de taper le nombre d'éléments affectés pour confirmer. Ce jeu de données est tout l'actif du projet — le panneau admin devrait rendre difficile de l'abîmer même quand je suis fatigué et que je vais vite.

L'API publique

Commodité et recherche, gratuit à tous les paliers. Rien ici n'est jamais vendu.

18.1 Points d'accès

Toute l'API est en phase 4. Dedans, les points d'accès basés sur des règles fonctionnent dès qu'elle existe ; les deux qui ont besoin d'un modèle attendent le §11.

```
GET /v1/search          q, region, scope, unit, script, limit, include_vulgar
GET /v1/corpus-items/:id enregistrement complet avec tags, tokens, liens, et traductions
POST /v1/normalise      { text } -> { canonical, script, rule_version }
POST /v1/transliterate  { text, from, to } -> { text, ambiguous[] }
POST /v1/lookup         { text, region?, include_vulgar? } -> formes attestées + traductions
                        exact / normalisé / trigramme. Pas de ML. (§14.2)
GET /v1/lexicon/:entry_id toutes les variantes régionales
GET /v1/origins/:entry_id étymologie, là où elle est enregistrée
GET /v1/coverage        statistiques publiques

    en attente du travail de modèle reporté (§11.3) :
POST /v1/classify       { text } -> { regions[], confidence, alternatives[] }
POST /v1/memory         { text } -> correspondances sémantiques floues ; a besoin d'embeddings
```

18.2 Paliers d'accès

Tout est gratuit. Les paliers existent pour garder l'API utilisable sous un trafic réel, pas pour la faire payer.

PALIER	LIMITES	POUR
Anonyme	60 requêtes/heure, recherche et lecture d'éléments de corpus seulement	jeter un œil
Clé gratuite	1000 requêtes/jour, tous les points de lecture	chercheurs, projets personnels, créateurs d'outils
Recherche / usage intensif	illimité, sur demande	usage académique, partenaires institutionnels, quiconque a une vraie raison de demander plus que ce que donne le palier gratuit — accordé à la main, toujours gratuit

Les chiffres ci-dessus sont une estimation de départ, pas une limite mesurée. Les vraies limites de débit et les plafonds de téléchargement dépendent de ce que le VPS peut vraiment encaisser, et je ne peux pas le dire honnêtement avant que l'hébergement ne soit tranché (§2.1). Ils seront ajustés contre la vraie machine une fois qu'elle existera.

18.3 Règles

- **Il n'y a pas de palier payant et il n'y en aura jamais.** L'API n'est pas un plan de revenus déguisé en limiteur de débit. C'est un effort à but non lucratif — les paliers existent pour protéger le serveur d'être martelé, pas pour monétiser l'accès. Une entreprise qui veut construire un produit payant par-dessus ces données est libre de le faire ; je ne suis pas celui qui la fait payer pour ce privilège.
- **Le corpus en masse n'est jamais verrouillé.** Cette ouverture gagne la bonne volonté dont tout le reste dépend.
- Chaque réponse porte le `snapshot_version` et le `rule_version` contre lesquels elle a été calculée.
- Limitation de débit au niveau du reverse proxy, pas dans le code applicatif.

Export et publication

Les formats que chaque public utilise déjà, et ce qui accompagne chaque publication.

19.1 Formats

FORMAT	POUR	REMARQUE
JSONL	général, entraînement de modèle	un élément de corpus par ligne, tags imbriqués
CoNLL-U	outillage linguistique	au niveau token, le standard que tout le monde en TAL lit déjà
TMX	industrie de la localisation	Choisi exprès — c'est ce que les outils de mémoire de traduction consomment déjà, donc aucune friction d'adoption pour le public commercial.
CSV	enseignants, journalistes, utilisateurs non techniques	aplatis et avec perte, et étiqueté comme tel
HF Datasets	l'écosystème ML	versionné, avec une fiche de jeu de données

19.2 Paliers d'export par droits

full	tout, usage recherche, non commercial
clean	contributions consenties seulement, utilisable commercialement
regional-*	tranches par région
lexicon	entrée de lexique / variante / forme seulement, pas de texte source
etymology	entrée de lexique, origine, forme source – petit, autonome, et probablement la tranche la plus citable disponible tôt

19.3 Chaque publication est livrée avec

Un numéro de version, un checksum, les comptes d'éléments de corpus par région et par unité. La version de règle utilisée pour les formes canoniques. Des statistiques d'accord, rapportées honnêtement. Un journal des changements contre la version précédente. Une fiche de jeu de données couvrant la méthode de collecte, les biais connus, les usages prévus et non prévus. Un DOI Zenodo.

Séquence de construction

Cinq phases, et une règle sur là où ce projet a le droit de s'arrêter.

OÙ EN EST VRAIMENT LA PHASE 1, À CETTE RÉVISION

Le schéma, le moteur de tâches, le consensus, et le pipeline d'ingestion des éléments de corpus sont construits et testés contre une base de données en fonctionnement. Rien de ce qui fait face à l'utilisateur ne l'est — pas d'interface de file de relecture, pas encore de scraper contre une vraie source. La checklist ci-dessous marque chaque ligne plutôt que de reformuler ce partage en prose.

Phase 1 — Pipeline central · but : une boucle d'annotation qui fonctionne

- Squelette du monorepo : `control-plane/` (backend + core sous `backend/core`, `worker`, `scrapers`), `data-plane/`, `intelligence-plane/nlp`, `frontend`, `shared`, `db` — **fait**. `worker/` est un vrai paquet avec `pg-boss` branché et une tâche en fonctionnement ; `scrapers/` est amorcé comme paquet mais ses dossiers `pipeline/` et `sources/` sont encore vides (§10.0)
- Schéma Prisma : `documents`, `corpus_items`, `tokens`, `links`, `tags`, `tasks`, `users`, `sources`, `audit_log` — **fait**, et il couvre maintenant aussi chaque table des phases 2/3/5 (lexique, traductions, corrections, signalements, instantanés, publications, outbox, objets stockés) — moins cher de déclarer une table vide maintenant que d'en migrer une plus tard
- `lexicon_origins` et `origin_enum` dans le schéma dès la première migration, même si rien n'y écrit avant la phase 2 (§7.6) — **fait**
- Migrations — **fait** ; données de seed écrites à la main pour les quatre régions — **pas commencé** (`db/seeds/` est encore un squelette vide)
- Auth : connexion relecteur, cookies de session, chaîne de guards — **partiellement fait**. Email/mot de passe avec `argon2id`, `JWT` `access+refresh` avec rotation et détection de réutilisation, double-soumission `CSRF`, et la chaîne complète `ThrottlerGuard` → `AuthGuard` → `RolesGuard` → `TrustGuard` sont construits et vérifiés en fonctionnement (§9.5). `GitHub OAuth` n'est pas encore branché.
- Moteur de tâches : réclamer, compléter, rejeter, reprendre ; règles de génération ; clés d'idempotence dès le départ — **fait**, et les complétions `review/region_tag/confirm` sont vérifiées de bout en bout en fonctionnement (§8.1). Les complétions `translate_*`, `transliterate_*`, `standardise`, `adjudicate`, `link_lemma` ont encore besoin des modules lexique/traduction.
- Routage d'adjudication et `target_regions` — dès la phase 1, pas plus tard. La règle de routage protège le corpus, et l'ajouter après coup une fois que les relecteurs ont pris leurs habitudes est pire qu'inutile. **La logique de routage elle-même est construite** (§8.5) et se déclenche automatiquement sous le seuil d'accord ; l'*interface* de la phase 1 est le panneau admin, qui n'existe pas encore — l'écran d'adjudication dédié arrive en phase 3.
- Interface de file de relecture, entièrement pilotée au clavier — **pas commencé**
- `CorpusItemText`, `RegionPicker`, `TaskCard`, `TagChip` — **pas commencé** (§15.3)
- Un scraper contre une source forum, de bout en bout — pas YouTube (§12.1) — **pas commencé** ; les fonctions du pipeline d'ingestion qu'il traversera (nettoyage, filtre de longueur, filtre de marqueurs, quasi-doublons, découpage en éléments de corpus, calcul du `match_key`) sont déjà écrites dans `control-plane/backend/core/src/ingestion/` et `text/`, juste pas encore pilotées par un vrai scraper
- Calcul du consensus + marquage pour adjudication — **fait**, vérifié en fonctionnement
- `pg-boss` avec intervalle de sondage et concurrence plafonnés dès le premier commit (§10.3) — **partiellement fait** : `pg-boss` est branché dans `worker/` contre `DIRECT_URL`, une tâche en fonctionnement (`expire_claims`, 5 min) ; le reste de la liste de tâches du §9.3 et les plafonds de sondage/concurrence eux-mêmes restent à faire
- `intelligence-plane/nlp` — `FastAPI`, endpoint health seulement, conteneurisé, dans `compose` — **fait** (§11)
- `data-plane` — `Actix`, endpoint health seulement, conteneurisé, dans `compose` — **fait** (§7.11)
- Les deux joignables depuis le backend, conteneurisés et avec health-check — **fait** ; affichés sur un widget de l'aperçu admin — **pas commencé**, puisque l'aperçu admin lui-même n'existe pas encore (§17.1)
- Docker Compose faisant tourner tout ça en local — **fait** (`docker-compose.local.yml` pour l'infra de dev, `docker-compose.prod.yml` pour la pile complète)

LA PHASE 2 NE SE TERMINE PAS AVEC UN JEU DE DONNÉES

L'objectif évident de la phase 2 serait « quelque chose de citable existe » — une page de téléchargement et un DOI. Le bon artefact pour un chercheur, le mauvais pour tous les autres, et ce sont tous les autres dont ce projet a besoin. Personne qui arrive sur ce site ne télécharge un JSONL. Il tape un mot dans le traducteur. S'il répond, il pourrait contribuer. Si la page d'accueil est un lien de téléchargement et un essai de méthodologie, il part.

Chaque problème difficile du §5 — recrutement de relecteurs, équilibre régional, rester en vie — est un problème de faire venir les gens et de les faire revenir. Un jeu de données ne peut pas faire ça. Un outil qui répond à une question qu'ils avaient vraiment le peut. Donc la phase 2 livre le traducteur, la vue de comparaison et la page de couverture comme un site qui fonctionne. L'instantané publié est un sous-produit, pas le but.

Phase 2 — Un site qui fonctionne · but : quelque chose qu'un Tunisien peut vraiment utiliser

La phase 2 à proprement parler n'a pas commencé — les lignes ci-dessous sont encore le plan, sauf trois morceaux d'infrastructure construits tôt parce que la séquence de construction de la phase 1 en avait besoin de toute façon.

- ☐ Lexique : `lexicon_entries`, `lexicon_variants`, `lexicon_variant_regions`, `lexicon_forms` — le traducteur tourne dessus. Schéma fait (§7.5) ; aucun module, aucun point d'accès encore.
- ☐ Interface d'origine des mots sur la page de l'entrée de lexique, plus la page `/origins` de navigation par strate
- ☐ Page publications, amorcée avec les travaux déjà cités aux §1.2 et §13.1
- ☐ Flux de liaison de lemmes et écran de relecture — travail d'annotation central
- ☐ **Page traducteur** : recherche de mots et de phrases, filtrée par région, honnête sur la couverture (§14)
- ☐ `pg_trgm` + normalisation `match_key` en fonctionnement (§13.5) — le traducteur n'a rien sur quoi tourner sans ça. `match_key` est déjà calculé sur chaque élément de corpus contribué (module `corpus-items` du §8.1) ; le côté lexique n'est pas encore construit.
- ☐ Journal des échecs du traducteur alimentant les données de lacunes
- ☐ Navigation et recherche publiques sur tous les axes de filtrage
- ☐ Pages permalien d'entrée avec historique des révisions et bouton de signalement (§16.4) — plus une simple vue en liste pour que je voie et résolve les signalements, puisqu'un bouton de signalement que personne ne regarde n'est pas un plan de modération. La file unifiée complète fusionnant les signalements avec les corrections, origines et publications (§17.2) est une gentillesse de phase 3, pas le minimum de la phase 2.
- ☐ Vue de comparaison à travers les quatre régions
- ☐ Tableau de bord de couverture, public
- ☐ Pages méthodologie, guide de style, licence, citation
- ☐ Interface de stockage de fichiers branchée : URLs présignées contre R2, dev utilise le disque local (§9.5) — **construit tôt** comme interface plus les deux adaptateurs (`control-plane/backend/core/src/storage/`) ; pas encore appelé depuis une route
- ☐ Pipeline d'export : JSONL + HF Datasets, avec checksum et versionné, servi via des URLs présignées sans passer par le backend
- ☐ Premier instantané publié avec une fiche de jeu de données et un DOI Zenodo
- ☐ Révocation de consentement + routine d'anonymisation, testée (§4.3)
- ☐ Chiffrement des sauvegardes branché avant que la première vraie sauvegarde ne tourne, pas après (§2.3) — **construit tôt** : `devops/backup/pg-backup.sh` chiffre en gpg avant que quoi que ce soit ne quitte la machine
- ☐ CI : `master.yml` (lint/typecheck/test/dérive de migration/CodeQL) et `verify-backup-restore.yml` — **construits tôt**, puisque la CI doit exister avant qu'il n'y ait quoi que ce soit qui vaille la peine d'être protégé avec elle ; `infra-watch.yml` toujours pas construit
- ☐ **Trancher l'hébergement et le stockage objet ici, pas avant** (§2.1, §2.5) — dimensionner contre l'usage mesuré, déployer, TLS, sauvegardes en fonctionnement et testées en restauration

Phase 3 — Communauté · but : de la contribution qui n'a plus besoin de moi

- ☐ Flux de contribution anonyme avec capture de consentement
- ☐ Page confirmer / voter — celle à friction minimale qui convertit les visiteurs
- ☐ Boîte de tâches pour les contributeurs, filtrée par niveau de confiance
- ☐ Interfaces de tâches de traduction et de translittération

- ☐ Propositions de correction + flux de relecture, partageant la file avec les propositions d'origine et de publication
- ☐ Classement par contributions approuvées et par région
- ☐ Page des lacunes guidée par les vrais déficits de couverture et les échecs du traducteur
- ☐ Tâche de calcul des niveaux de confiance
- ☐ Tableau de bord admin : aperçu, utilisateurs, sources, tâches, instantanés, opérations de confidentialité

Phase 4 — Couche de service et assistance automatique · seulement une fois que le corpus le mérite

- ☐ Moteur de règles de standardisation + régénération versionnée, avec retour arrière via un `rule_version` antérieur (§17.2)
- ☐ Embeddings via API hébergée + pgvector ; récupération floue en mémoire de traduction
- ☐ API publique, clés, limitation de débit, tableau de bord d'usage
- ☐ Documentation de l'API
- ☐ Formats d'export TMX et CoNLL-U
- ☐ **Pré-filtre de dialecte** — inférence par lots sur GPU serverless, hors machine (§11.3)
- ☐ **Classificateur régional** entraîné sur le corpus lui-même — la contribution de recherche
- ☐ Registre de modèles et évaluation contre des splits figés
- ☐ PostHog (palier gratuit, §10.0) — je ne peux pas savoir si le traducteur est utilisé sans ça, et réutiliser un outil que je connais déjà bat réévaluer les plateformes d'analytics depuis zéro

Phase 5 — Voix et génération · le plus lointain, et vraiment optionnel

- ☐ Audio d'attestation : enregistrer un extrait d'une variante, le rejouer sur l'entrée de lexique (§7.11)
- ☐ Tables `speakers` et `media` peuplées pour la première fois
- ☐ Stockage objet re-choisi sur le coût d'egress, pas le coût de sauvegarde (§2.5)
- ☐ Application du consentement voix et chemin de suppression (§4.3)
- ☐ Génération ancrée pour les entrées non attestées du traducteur, clairement étiquetée comme générée (§14.6)

Ce que je ne construis pas

audio au-delà des extraits d'attestation de la phase 5 — pas de reconnaissance vocale, pas de synthèse vocale, pas de modèles de parole

annotation morphologique

tout modèle avant la phase 4

inférence sur serveur

applis mobiles

paliers payants

sous-régions

Tout est prévu dans le schéma — colonnes nullable, tables squelettes, les structures `is_machine` et `model_versions` déjà en place — donc ajouter n'importe lequel de ces éléments plus tard est une migration, pas une réécriture.

LA RÈGLE À LAQUELLE JE ME TIENS

Les phases 1 et 2 sont dimensionnées pour produire, à elles seules, quelque chose de complet, publiable, citable. **Si ça s'arrête après la phase 2, ça aura quand même produit quelque chose qui n'existait pas avant.** Tout ce qui vient après, c'est du bonus, pas une condition pour que le travail en ait valu la peine.

OpenDerja_TN · Firas Lamouchi · planification commencée le 14 avril 2026, développement commencé le 28 mai 2026, mis à jour le 25 août 2026 après le transfert du dépôt vers un hébergement GitHub distant
Documents compagnons : Architecture & Engineering Blueprint · VOICE.md